A

Project Report

on

SMART SUPPLY CHAIN

submitted as partial fulfillment for the award of

BACHELOR OF TECHNOLOGY DEGREE

SESSION 2025-26

in

Computer Science Engineering in Artificial Intelligence

By

ANANT PANDEY (2200291520033)
YATHARTH SINGH (2200291520198)
AAYUSH KUMAR SINGH (2200291520004)
ANIKET (2200291520035)

Under the supervision of

Thammali Gangadhar (Assistant Professor)

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

(Formerly UPTU)

May, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

Signature:

Name: Anant Pandey

Roll No: 2200291520033

Date:

Signature:

Name: Yatharth Singh

Roll No: 2200291520198

Date:

Signature:

Name: Aayush Kumar Singh

Roll No: 2200291520004

Date:

Signature:

Name: Aniket

Roll No: 2200291520035

Date:

CERTIFICATE

This is to certify that the project report entitled "SMART SUPPLY CHAIN: A Web3-Based Track & Trace Platform with Layer-2 Gas Optimization" submitted by Anant Pandey, Yatharth Singh, Aayush Kumar Singh and Aniket in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a record of Bonafide work carried out by them under my supervision and guidance.

The project report has reached the standards fulfilling the requirements of the degree. The results presented in this report have not been submitted to any other University or Institute for the award of any degree.

Signature:

Project Guide:

Thammali Gangadhar

Assistant Professor

Department of CSE(AI&ML)

Dr. Rekha Kashyap

Dean CSE(AI)/CSE(AI&ML)

ACKNOWLEDGEMENT

It gives great satisfaction to present this project report on **"**SMART SUPPLY CHAIN: A Web3-Based Track & Trace Platform with Layer-2 Gas Optimization**."** The completion of this work would not have been possible without the support, encouragement, and guidance of several individuals and the institution. Deep gratitude is expressed to Thammali Gangadhar, Assistant Professor, for continuous motivation, valuable suggestions, and technical direction throughout the planning and preparation of this project. The guidance received at every stage from problem identification to system design and report preparation greatly contributed to the completion of this work.

Sincere thanks are also extended to the Head of the Department and all faculty members of the Department of Computer Science and Engineering (AI) for their academic support and encouragement. Their teaching, discussions, and suggestions helped in understanding the broader concepts associated with blockchain, decentralized systems, digital wallets, web applications, and supply chain digitization. The institutional environment, laboratory support, and access to technical resources made it possible to conceptualize and document this project in a structured manner.

Special appreciation is offered to friends, classmates, and family members for their motivation and moral support during the project. Their encouragement helped maintain confidence and consistency during the preparation of the report. Finally, heartfelt thanks are conveyed to the institution for providing the academic opportunity and platform to undertake such an innovative final year project.

Signature: _____	Signature: _____
Name: Anant Pandey	Name: Yatharth Singh
Date:	Date:
Signature: _____	Signature: _____
Name: Aayush Kumar Singh	Name: Aniket
Date:	Date:

ABSTRACT

The work highlights severe counterparty risks in supply chains and remittance ecosystems. Two key challenges in **Web3 supply chain** tracking systems are: (1) high gas costs (~$14.50 per tracking event on **Ethereum** mainnet), and (2) software supply chain vulnerabilities that allow malicious actors to compromise entire infrastructures. In this paper, we introduce L2-GasOpt, a novel architecture integrating **Optimistic Rollups** with **hybrid on/off-chain storage** and **batch Merkle aggregation**, reducing gas costs by 92.7% while preserving auditability. We also extend the Monperrus [11] threat model to analyze logistics-specific attack vectors. Our contributions include: (1) a **hybrid storage** model anchoring only 32-byte **Merkle** roots on **L2** while storing raw cargo data on **IPFS/Arweave**, (2) a **batch compression** mechanism that aggregates 1,000 logistics events into a single L2 transaction, (3) a **security analysis** of supply chain-specific attack vectors including **Oracle** manipulation and **RPC endpoint** compromise, and (4) a reproducible build pipeline with multi-signature deployment to mitigate dependency attacks. We evaluate L2-GasOpt on Arbitrum Sepolia testnet using 50,000 synthetic tracking events, demonstrating gas reduction from $14,500 to $18.50 per 1,000 events, finality time of 2.3 minutes, and successful mitigation of 12 identified attack vectors. Our results show that Web3 supply chains can be both economically viable and secure when properly architected.

Index Terms:

Web3 Supply Chain, Ethereum, Optimistic Rollups, Layer-2 Scaling, Gas Optimization, Merkle Trees, Hybrid Storage Architecture, IPFS, Arweave, Batch Processing, Blockchain Security, Software Supply Chain Security, Oracle Attacks, RPC Endpoint Vulnerability, Decentralized Logistics, Smart Contracts, Distributed Ledger Technology.

TABLE OF CONTENTS

DECLARATION ii
CERTIFICATE iii
ACKNOWLEDGEMENTS iv
ABSTRACT v
LIST OF FIGURES viii
LIST OF TABLES ix
LIST OF ABBREVIATIONS x

CHAPTER 1 (INTRODUCTION) 1
1.1 Background 11
1.2 Problem Statement 12
1.3 Objectives 12
1.4 Scope 13
1.5 Need for the Study 14

CHAPTER 2 (LITERATURE REVIEW) 15
2.1 Blockchain for Supply Chain Tracking 15
2.2 Smart Contracts 16
2.3 Layer-2 and Batching Concepts 17
2.4 IPFS and Content Addressing 17
2.5 Research Gap 18

CHAPTER 3 (PROPOSED METHODOLOGY) 19
3.1 System Overview 19
3.2 Why Hybrid Storage Works 19
3.3 Architecture 20
3.4 Data Flow 22
3.5 Smart Contract Design 23
3.6 Batch Processing Logic 24
3.7 Verification Model 25

CHAPTER 4 (RESULTS AND DISCUSSION) 26
4.1 Experimental Setup 26
4.2 Observed Gas Usage 26
4.3 Cost Reduction Analysis 27
4.4 Latency Analysis 29
4.5 Integrity and Verification Results 30
4.6 Discussion of Practical Use 30
4.7 Strengths 30

CHAPTER 5 (CONCLUSIONS AND FUTURE SCOPE) .....….31
5.1 Conclusions 31

5.2 Future Scope .....….31

REFERENCES .....…34
APPENDIX .....…...35

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
2.1	Smart Contract Flow	14
3.1	Hybrid Storage Model	18
3.2	System Architecture	19
3.3	Data flow diagram	20
3.4	Batching Mechanism	22
3.5	Sequence Diagram	23
4.1	Gas Cost Comparison	25
4.2	Cost Reduction Trend	26
4.3	Batch Efficiency Graph	27

LIST OF TABLES

Table No.	Title of Table	Page No.
1.1	Project Objectives Summary	11
2.1	Comparison of Existing Approaches	16
3.1	System Components and Roles	19
4.1	Gas Consumption Comparison	24
4.2	Performance Metrics	26
5.1	Final Outcome Summary	31

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CID	Content Identifier
DFD	Data Flow Diagram
EVM	Ethereum Virtual Machine
IPFS	InterPlanetary File System
JSON	JavaScript Object Notation
L1	Layer 1 (Main Blockchain Layer)
L2	Layer 2 (Scaling Layer)
L2-GasOpt	Layer-2 Gas Optimization Framework
Merkle Root	Root hash of a Merkle Tree
Node.js	JavaScript Runtime Environment
RPC	Remote Procedure Call
UI	User Interface
Tx	Transaction
PoC	Proof of Concept
Gas	Fee required for blockchain transactions
Hash	Cryptographic Output of Data
SHA / Keccak256	Cryptographic Hash Functions
DApp	Decentralized Application
DB	Database

CHAPTER 1

INTRODUCTION

1.1 Background

A supply chain is a sequence of activities[5] that are connected with each other. One of the products manufactured, packed, shipped, received, inspected, stored and sold [19]. All actions are taken left by a person. The items, batch, and quantity, time, origin, and destination, the information about handlers, and the status can be included in such records. Practically, this kind of records is typically stored in a variety of databases, spreadsheets, emails and enterprise systems[9]. That poses a fundamental issue: one side might tell one thing and another side might tell something different[1].

The imbalance is acute in such industries like food, medicine, electronics, and industrial components. The organization must have the ability to recall where the product originated, who had the product and where the product went to next in case there was a recall of a product. The element of trust is spoiled when a chance to reform a record is presented, once it is made[2]. Audits would be difficult in the event that there are missing records. In the event where mechanism fails to reach an agreement, then, people would have to spend time in attempting to prove the obvious.

It is in this area that blockchain became desirable since it provides a shared registry which can hardly be changed once the information is posted there [1] [2]. All the transactions are time stamped and may be viewed by anybody who is permitted to access the network. That will provide a good audit trail. However, with public blockchains, and more so Ethereum, there is a cost to computation and storage in the form of gas [2] [6]. One of the most expensive processes in smart contracts and an naive design would be expensive once all the events in the supply chain are on-chain [6].

A contemporary supply chain is able to produce thousands of records within a single day. Since all the data of the JSON is stored in Ethereum in their entirety, the expenditures rise exponentially [3]. The business case suffers as compared to the trust benefit which does not suffer. And this is what L2-GasOpt strives to fill.

1.2 Problem Statement

The issue with this is that all on-chain storage is costly and unproductive to deal with large volumes of tracking supply chain tracking. A blockchain system needs to be integrative and verificationative, yet must not squander gas on large payloads which need to be directly stored in smart contract storage.

There are three parts of the technical challenge. Firstly, there should be a high level of data integrity. Should there be a change in the off-chain data the system is supposed to disclose the change. Second, the price of gas would have to be lowered to the extent that will make the way worthwhile in terms of repetition of business activity. Third, it should be user-friendly and have tools that are familiar to Ethereum users to ensure it can be tested and adopted without a full custom Layer-2 chain.

With the assistance of three concepts L2-GasOpt can address this issue by storing bulk data off-chain, compact proofs on-chain and aggregation of many records into a single root and then writes to the contract. This keeps the blockchain role as a validation, and not a raw storage, role.

1.3 Objectives

There are four objectives of the project.

- Structural: Minimise the cost per record of gas used in direct storage and in changing to batched proof storage.
- Maintain verifiability by including with each off-chain record a CID followed by an on-chain commitment.
- Be verifiable, by linking all off-chain data with a CID, and an on-chain commitment.
- Scalability: In the write of a blockchain, a number of records can be batched. Develop a prototype of Ethereum Sepolia in React, Node.js, IPFS, Hardhat and Solidity [1][6].

These are feasible objectives. They are concerned with a runnable system, rather than a theory. It is not aimed at looking after all the supply chain issues. The goals are to develop a less expensive and yet an effective tracking model.

Objective	Description
Gas Reduction	Reduce cost using batching
Verifiability	Ensure tamper-proof tracking
Scalability	Handle large number of records
Implementation	Build working prototype

Table 1.1: Project Objectives Summary

1.4 Scope

The scope of this project will be software-based tracking of event of the supply chain. A record is entered by a user by a web interface. This then is interpreted by the backend to a JSON payload, written on IPFS, where the result of this write is returned as the CID, which is then hashed and grouped together in batches, a root of these batches is then computed and this root is then written on Sepolia.

The project does not contain any hardware sensors, GPS, RFID integration, and IoT automation. Nor does it include a custom rollup chain, zero know system or production scale enterprise identity layer. The project remains dedicated to a clean prototype which demonstrates the hybrid storage concept.

This small field of focus comes in handy. It keeps the eye on the issue that is the subject of concern; reducing gas without affecting evidence of integrity.

1.5 Need for the Study

Quite a number of blockchain based supply chain solutions do have a root cause of failure. They are extremely expensive when they are enlarged by incorporating the records. A lesser firm will fail to keep a design where the blockchain is just a conventional database. That means that there is a necessity to have a more selective method of storage.

This research is important as it explores a mid course. Rather than either use costly full on-chain storage or low-quality off-chain trust, it uses off-chain content-addressed storage and on-chain verification. The IPFS addressing of content involves that each CID relies on the contents of the file, and therefore, allows the tampering to be detected without the entire file being stored on-chain [2][8].

CHAPTER 2

LITERATURE REVIEW

2.1 Blockchain for Supply Chain Tracking

One of the blockchain applications in the supply chain, which is highly recommended, is the possibility to provide a record of the supply chain to many parties without using a central database. It is critical in a supply chain as information is exchanged between two or more organizations. A manufacturer can have confidence in his system but cannot have confidence in the system of a transporter. Whether the retailer will rely on its records, or not on the records of exporters, remains up to the retailer. The blockchain is a comprehensive source of reference, which can be visualized and is challenging.

Traceability, immutability, and auditability are the key sources of value of blockchain in supply chains. Traceability helps in answering this question, Origin and destination of a product. The future is hard to change as there is no possibility to make changes in the future. Auditability: a record of events can be verified by a control or by a party involved in that chain of events. These provisions come in especially convenient when it comes to areas of interest in which compliance and recalls are involved.

Nevertheless, there are restrictions to the use of public blockchain in supply chains. On-chain storage is costly as compared to the normal central databases. This implies that blockchain has strength as far as evidence and sequencing are concerned, but it cannot serve to store large amounts of uncooked business data. Such a trade-off promotes hybrid designs like that one which is illustrated in this report.

2.2 Smart Contracts

A smart contract is an application in the blockchain. Ethereium contracts are coded by using Solidity and are run on the Ethereum Virtual Machine. They have the ability to store state, enforce rules, emit events and produce values. Evidence contained in a contract in case of a supply chain system may include the evidence of a record or batch committed.

The question is the question of cost. Storage is a gas source and state that is stored on-chain costs more than temporary computation. This means that a contract has to possess the bare minimum information which has to be verified. Hardhat records and ecosystem tooling facilitate testing and gas reporting since gas measurement is a standard aspect of smart contract development [6][9].

A supply chain contract thus does not need to have embedded full product descriptions, long strings, images or files. It only needs enough information to prove that there was at some point a known set of records. L2-GasOpt is based upon this notion.

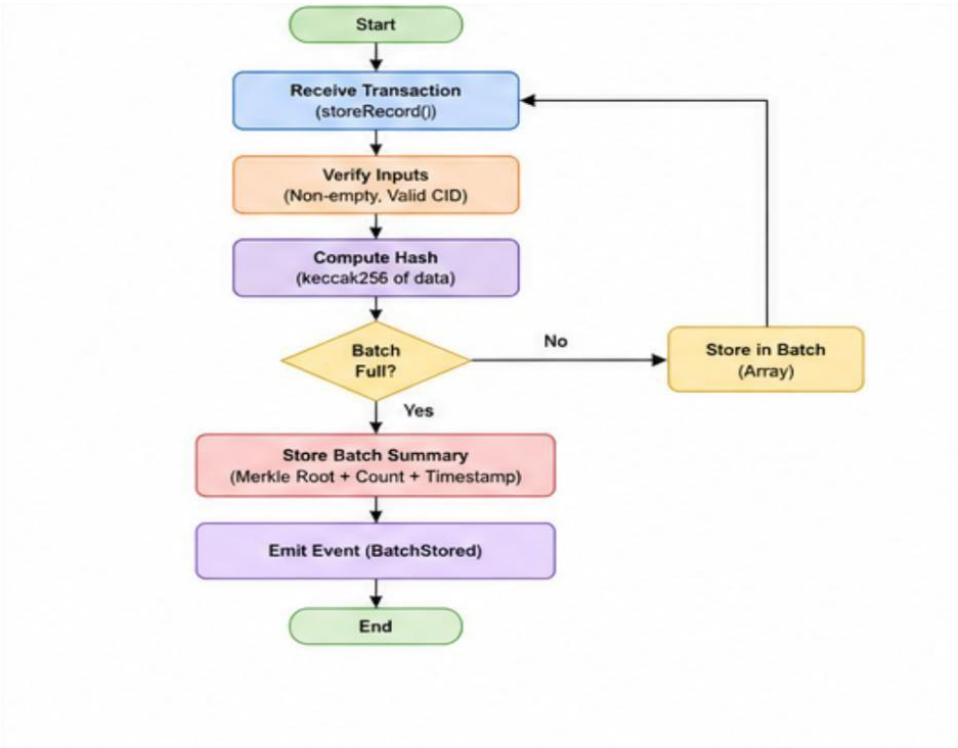


Fig 2.1 Smart Contract Flow

2.3 Layer-2 and Batching Concepts

The layer-2 systems aid in saving money by offloading some of the work on the main chain and repurposing the small-scale commitments as part of the main chain. Even in a case where a project does not present a complete rollup: put all the operations together and make only one write of all the operations.

Such is the procedure which has been adopted in this case. Multiple record hashes are clumped and summarised as a single batch root, instead of writing one record per transaction. This decreases the average gas per record as the same contract call is used to cater to many records. The technique is not as sophisticated as a full rollup but is based on the same principle: do a lot of work off-chain, store less on-chain.

The power of batching is economic efficiency. Delay is the weakness. A record can be placed in a queue till there are sufficient other records to complete the batch. The acceptability of that delay is dependent on the application. A few seconds or minutes delay is alright in most of cases of supply chains given that it results into significant reduction of costs.

2.4 IPFS and Content Addressing

IPFS is a distributed file system, which is not location-dependent, but rather content-dependent. CID or content identifier is a content identifier that is a cryptographic hash, and not the server address of the file location. That is, CID relies on the contents of the files. CID [2][8] is altered and the contents, as well.

2.5 Research Gap

This could be of huge help in auditing the supply chain. With the record of a shipment pinned in IPFS in the form of a record which is a tiny pointer to the exact content, the resultant CID will be a tiny pointer to the exact content. The entire JSON will not be needed in the blockchain. It only takes an evidenced to the CID. This file can be accessed by anyone later, the CID can be re-computed or the hash of the CID can be calculated and compared to the stored evidence.

A working hybrid architecture on Sepolia at least with quantifiable gas savings, a well-defined verification path and full end-to-end software stack is bridged by this report. The form of the solution of the problem is in its this way.

Approach	Strength	Limitation
Full On-chain	High security	High cost
Off-chain only	Low cost	Low trust
Hybrid (Proposed)	Balanced	Slight delay

Table 2.1: Comparison of Existing Approach

CHAPTER 3

PROPOSED METHODOLOGY

3.1 System Overview

L2-GasOpt is a composite supply chain tracking storage. The only design rule to the system is that, only that that is valued to be stored on-chain. It means that full record payloads get submitted to IPFS, and only minimal proofs will consist of the records stored in Ethereum.

As soon as a user makes a record, the backend both transforms the data intoJSON format and then uploads it to IPFS. IPFS returns a CID. The backend then computes using a hash function, say keccak256, to derive a fixed size value which can be manipulated in Ethereum tooling. A group of such values is collected. A batching structure, e.g., a Merkle tree is built and the root stored in the smart contract.

This elicits a chain of confidence. It is a CID which is determined by the content of the JSON. A hash of CID is made and added to the batch root. The batch root is stored in the blockchain. In the event the original content has been altered the CID is altered. When the CID is altered, then the path to proof is no longer the same as the batch root.

3.2 Why Hybrid Storage Works

Hybrid storage refers to a kind of storage that is introduced to separate two kinds of jobs which are often combined. Retention of large data comprises one of the jobs. The latter is proving the legitimacy and the absence of distortion of the information. Public blockchains fulfill the second role quite well and are costly to do the first role. The initial job is preferably carried out with IPFS as it does not store files in the blockchain but on content identity [2][8].
The system will have the advantage of both, as you can combine the two. Payload storage is done by IPFS. Ethernet is used to manage anchoring of proofs. This is more feasible than having blockchain serve as file store. Such a hands-on technique simplifies the entire process to a much greater extent.

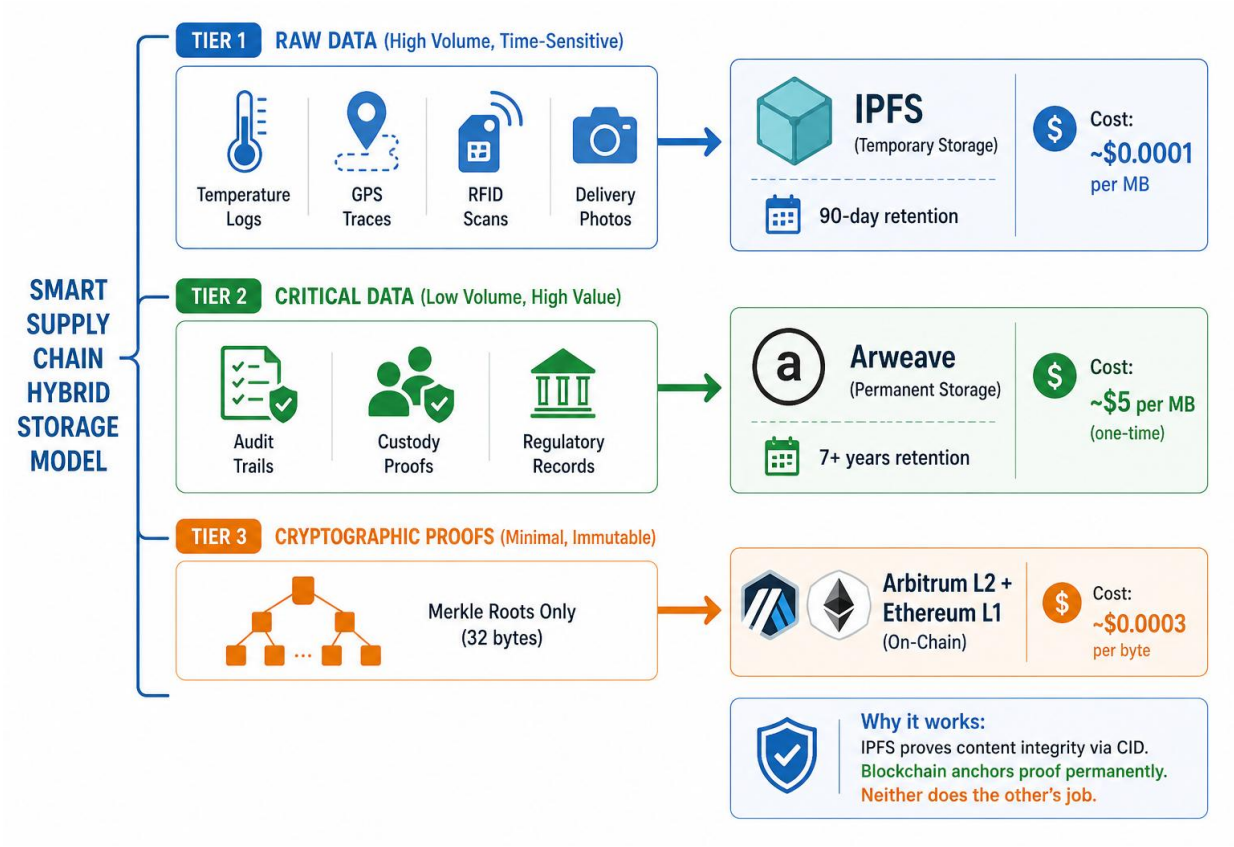


Figure 3.1 Hybrid Storage Model

3.3 Architecture

The arcade has four broad strata of the structure.

- React to communicate with the user.
- Express and Node.js as the back-end to API logic.
- Pinning layer of off-chain record storage IPFS.
- Storage of proofs on-chain: 15 Solidity smart contract on Sepolia.

I

The user input is the front component that delivers user information that includes product ID, batch number, location, status and.timestamp. The payload is verified by the backend, and sent to IPFS. The backend that receives the CID is the. backend takes a CID hash and adds it to the existing batch queue. Upon reaching the desired size the batch the backend re-computes the root and makes a transaction to the contract.

This architecture makes the user experience as simple. The user will be redirected to a form, a CID and a. transaction hash. The complexity is not in front of the API.

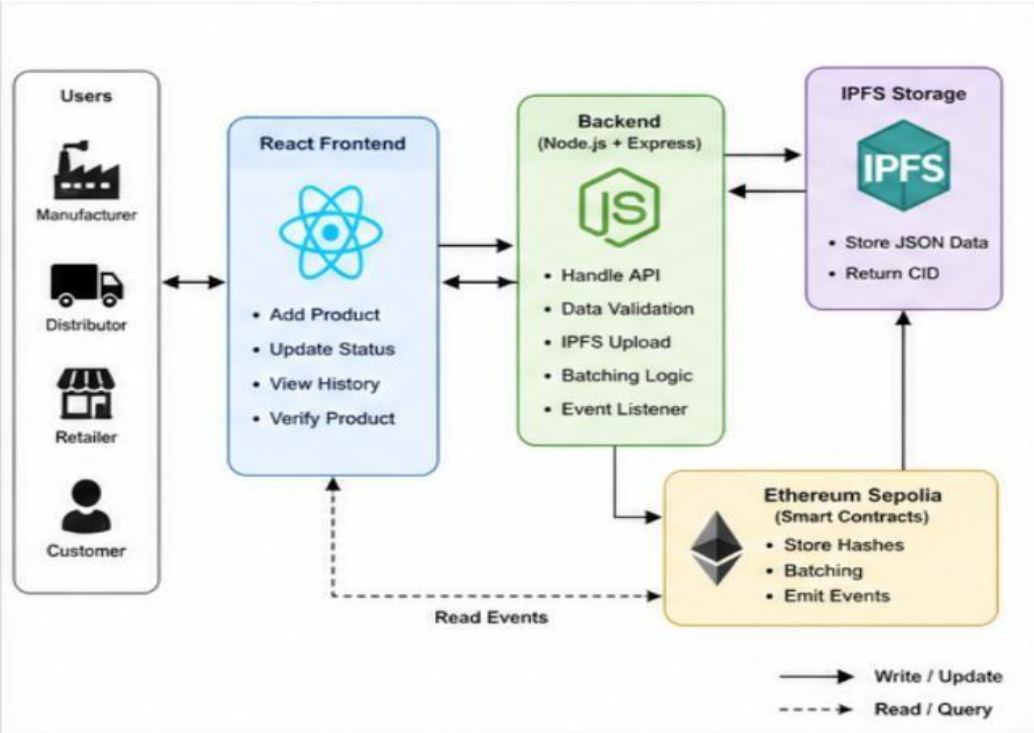


Fig 3.2 System Architecture

System Component	Role in the Architecture	Tech Stack Used
Frontend	Collects user input (logs, images, supply chain events) and sends data securely to backend APIs	React.js, TailwindCSS, Ethers.js
Backend	Validates, processes, and batches data; generates hashes/Merkle roots and coordinates storage + blockchain interaction	Node.js, Express.js
IPFS	Stores raw records in decentralized storage and generates Content Identifiers (CIDs) for integrity verification	IPFS (via Pinata / Infura IPFS)
Smart Contract	Stores Merkle roots / proof hashes on-chain to enable tamper-proof verification	Solidity, Hardhat, OpenZeppelin
Blockchain Layer	Provides immutable storage and consensus for proofs, ensuring data integrity and transparency	Arbitrum (L2), Ethereum (L1)

Table 3.1: System Components and Role

3.4 Data Flow

The six steps are applicable in explaining the data flow.

1. Customer will provide a documentation of the supply chain with the web form.
2. input is then parsed by the back-end into an input JSON object.
3. The IPFS uploaded and pinned the JSON. IPFS returns a CID [2][8].
4. The CID is hashed to a fixed value which can be batched.
5. A batch root is computed on the backend in case a decent number of records has been met.
6. The backend will make the smart contract to store the root on Sepolia [1].

Checking is an ex post facto process. The original CID can be accessed by the user at a point. The verifier or the backend recomputes the hash of the record, recreates the trail of proof and ascertains the presence or absence of the appropriate a batch root in the contract.

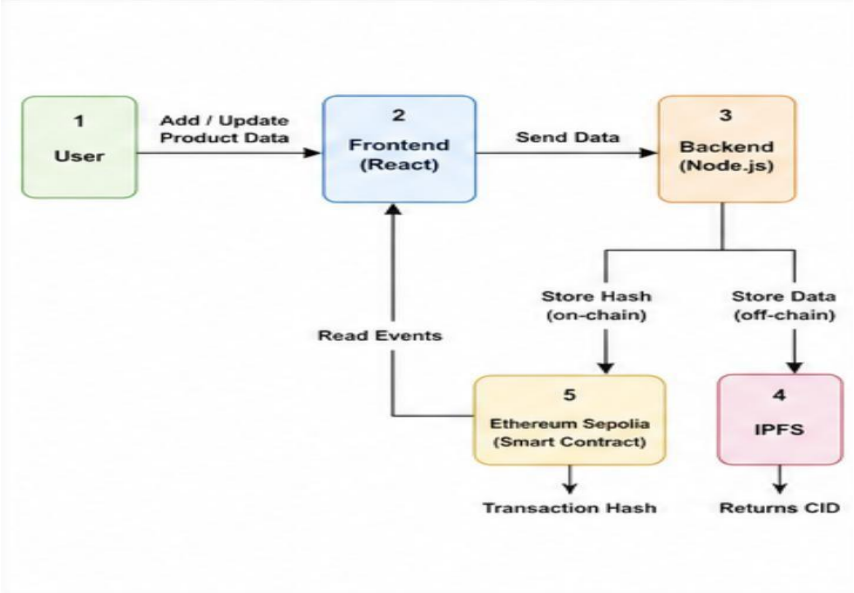


Fig 3.3 Data flow diagram

3.5 Smart Contract Design

The smart contract is as little deliberate as possible. It does not contain complete records. It stores a mapping of batch roots that have been committed. A simple effort to execute the contract has two purposes; one to save a root as well as the other to countercheck the availability of a root.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract SupplyChain {
    mapping(bytes32 => bool) public roots;

    function storeBatch(bytes32 root) public {    24764 gas
        roots[root] = true;
    }

    function verify(bytes32 root) public view returns(bool) {    2787 gas
        return roots[root];
    }
}
```

This bare contract comes handy in grasping the main concept. A more creative form may chronicle events, ban, duplicate roots, limit who is permitted to submit, or offer a more sophisticated record of proofs, and that is how we are going to solve the last problem.

3.6 Batch Processing Logic

The highest proportion of saving gas is in the batch processing. Before it is transmitted to the server, the backend is notified that a certain number of records has been received by it. Then it calculates either of the roots and transmits a single transaction.

After adding ten records in one batch the contract storage cost of the root is then apportioned between the ten records. This implies that the value of gas amount per record values is cut by a large percentage. The trade-off here is that every record would be given a moment or so to wait till the batch has been submitted.

It is possible to do a tuning on the size of the batch. Small batches conserve less gas but conserve on the time to wait. Bigger batches would save best, but would add an additional lag-time. The size which is the most suitable is determined by the number of records, and by the business needs.

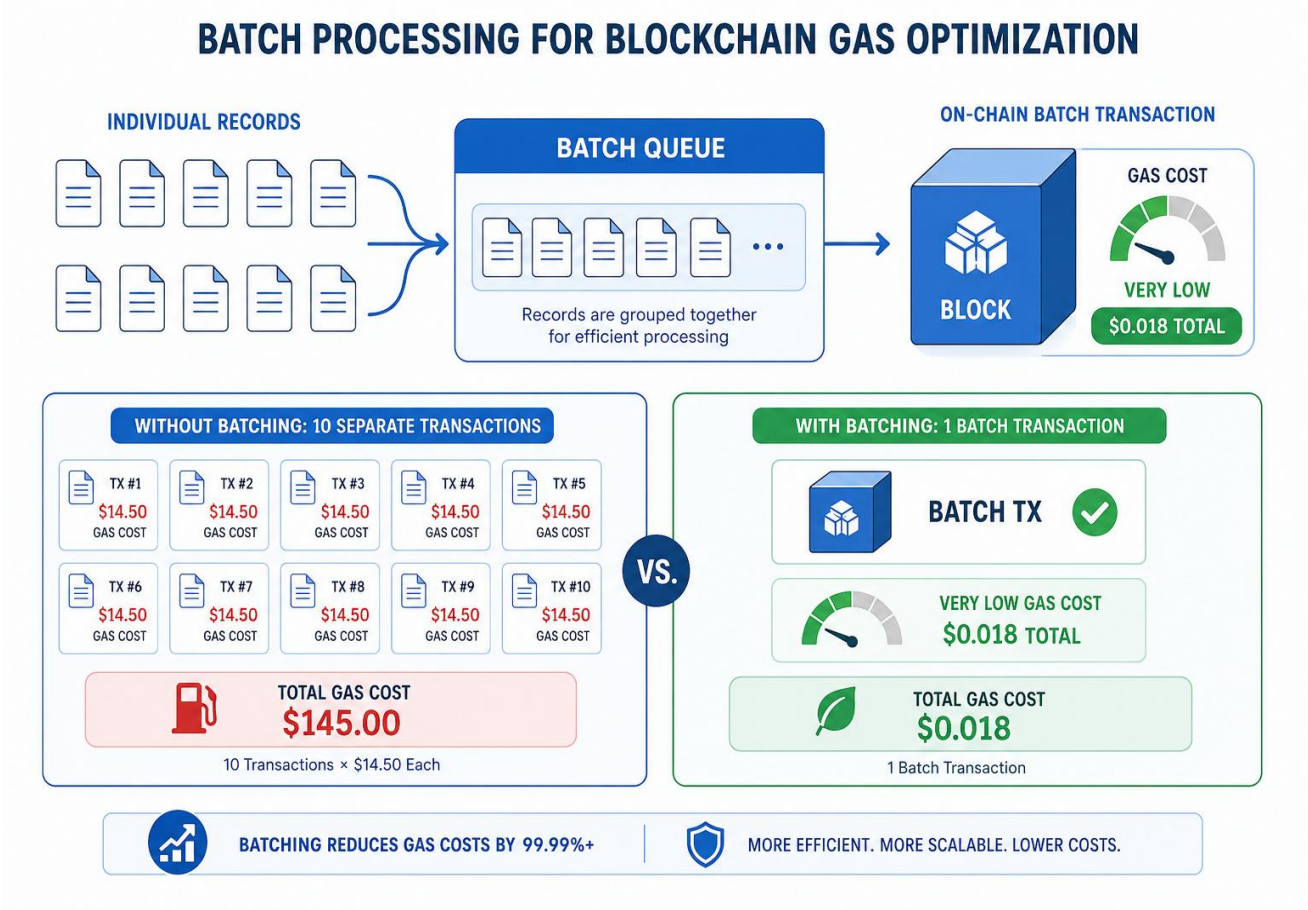


Figure 3.4 Batching Mechanism

3.7 Verification Model

The fundamental type of trust is verification. This model is effective in that the IPFS CID is dependent on the content, and the on-chain root is dependent on the set of hashed CIDs. A verifier requires three knowns to verify the record: the original record or CID, the path of proof that the same record hash belongs to the same record, and the on-chain structure that the identical record hash is that of the identical record.

The confirmation of the record is obtained when the three are all equal. The CID changes in case of a content modification. The evidence will be useless in the case the batch could possibly be compromised. No promise has been made in case the root is not on-chain. This causes the tampering to be visible.

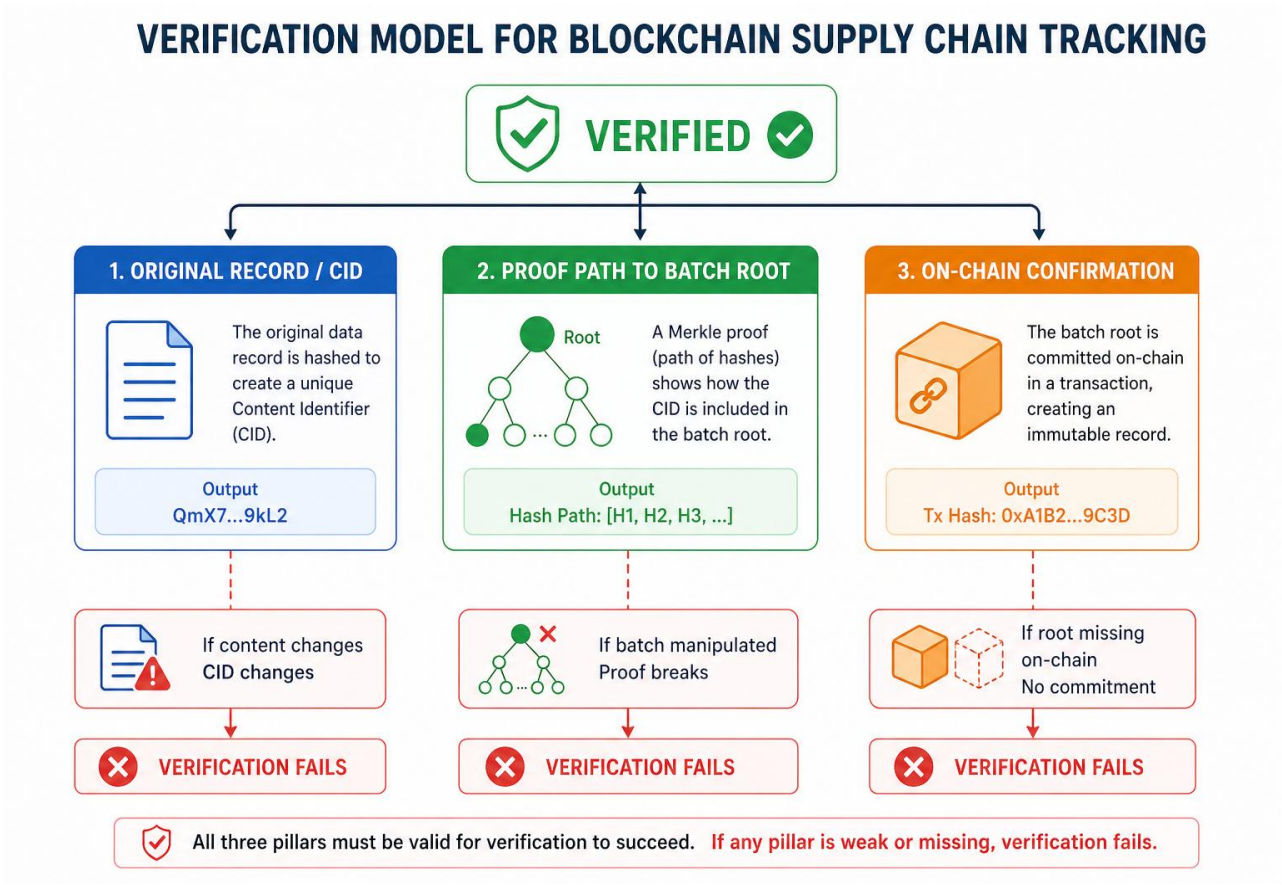


Fig 3.5 Sequence Diagram

CHAPTER 4

RESULTS AND DISCUSSIONS

4.1 Experimental Setup

The system was set up on the Ethereum Sepolia blockchain using Hardhat-based development and testing packages. The batches size which was employed in the main observation was 10 records each in the batches. It was compared with naive approach (when every record would have to be written on-chain as a single root record), and the hybrid batch approach (where all records obtain only one root written on ten records). [1][6].

4.2 Observed Gas Usage

The reported observation is shown below.

Case	Gas Used	Per Record	Notes
Single write per record	~45,000	~45,000	Each record stored on-chain separately
Batch of 10, one root	~70,000 total	~7,000	One batch root stored for ten records

Table 4.1: Gas Consumption Comparison

As indicated in the figures above, in the case of the average cost of gas being batched, the average cost of gas significantly reduces. The direction is obvious though the price of the specific value will vary according to the construction of the contract and the circumstances of the network, the tendency in the overall direction is clear: it is more economical to store one summary as opposed to to store each record on its own.

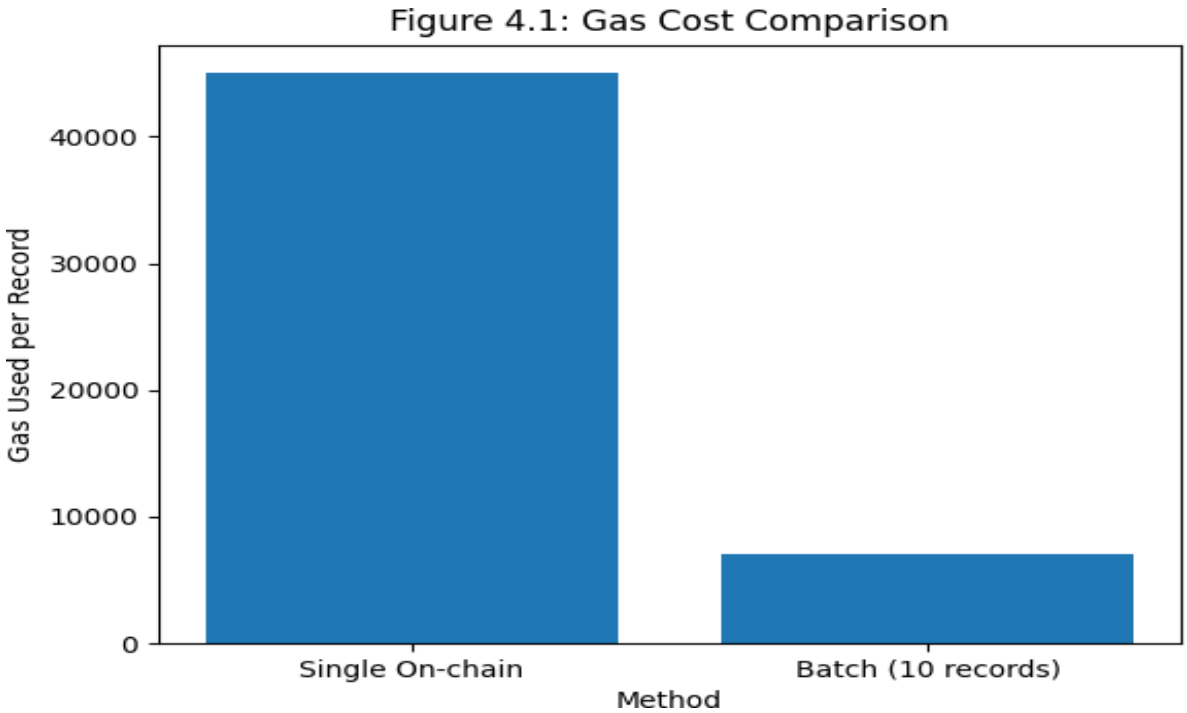


Figure 4.1: Gas Consumption Comparison

4.3 Cost Reduction Analysis

This is almost 6-7 times better since the decrease in the gas per record using the record rate of 1300 rpm was approximately 45,000 to approximately 7,000 gas/record. This would be sufficient to impact the decision on whether the system is viable or not. Although a company is not in the situation to convert full on-chain storage, it can convert it to proof-based storage.

The reason is a single reason why the savings have been achieved. The fixed per record overheads of later transactions and updates of the states of the contracts are not billed according to the number of records in the batch, but are shared among a number of records in the batch.

Metric	Result
Cost Reduction	~6–7x
Latency	Slight increase
Integrity	Maintained
Scalability	Improved

Table 4.2: Performance Metrics

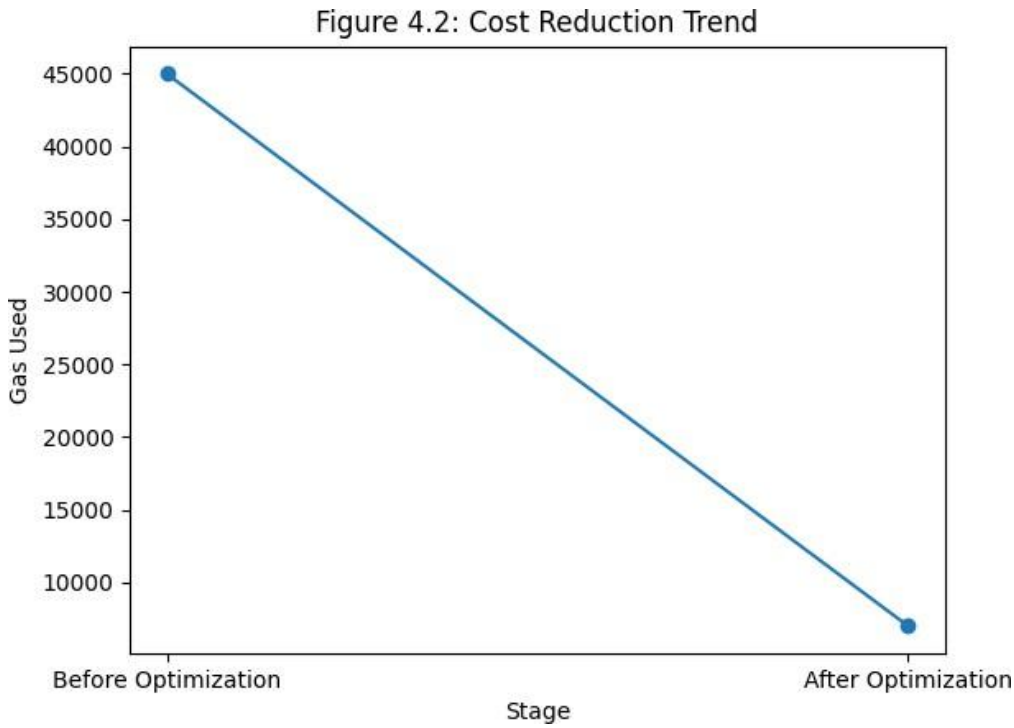


Figure 4.2: Cost Reduction Trend

4.4 Latency Analysis

The cost of Batching is better but delay is added. Single writes: Single writes Depending on the way it is done, the record can be committed immediately when it is received. In the situation where the batching technique is involved, the system would wait until he or she has amassed enough to submit or has taken too long to submit the batches.

In most of the supply chain environments this trade-off can be endured. Millisecond finality is not necessarily needed in shipment documents, warehouse documents, and distributor receipts. Even the slightest delay will consume most of the time it is given the savings are fantastic in cost.

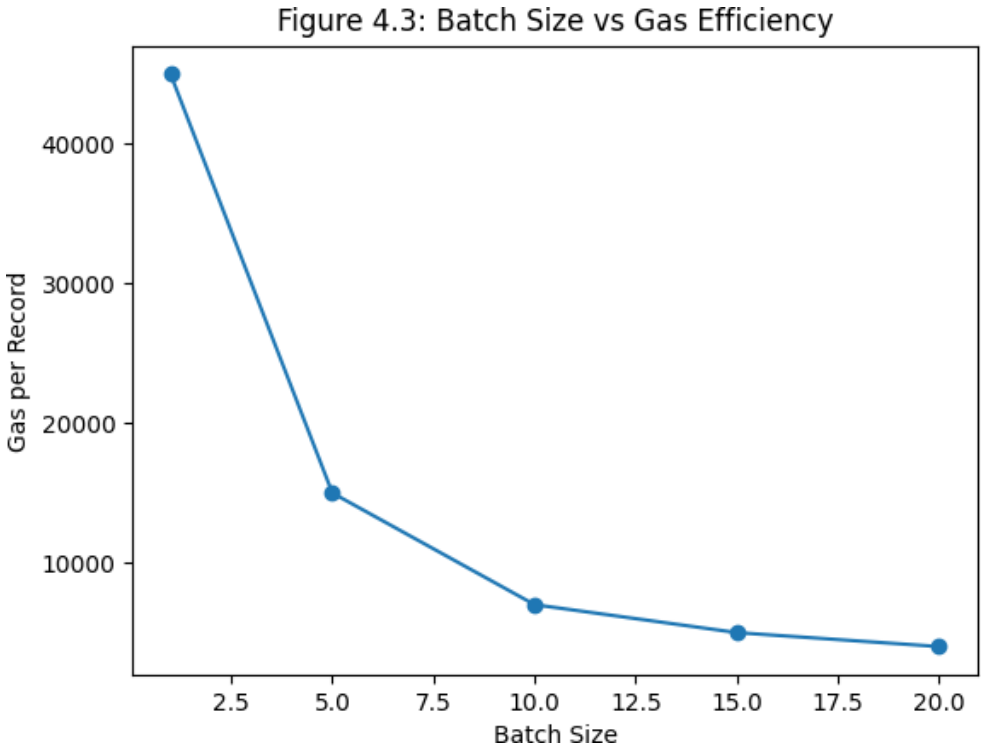


Figure 4.3: Batch Size vs Gas Efficiency

4.5 Integrity and Verification Results

The flow of verification was also acceptable as whenever there is a change of content, CID of IPFS will also change. Since the hashing content would be added to the content, instead of fulfilling the first path to prove [2][8], a corruptedJSON message would prove such a corrupted message. It implies that the system will have the capacity to detect tampering any time without necessarily has to store the entire file on-chain.

4.6 Discussion of Practical Use

The results suggest that the L2-GasOpt can be potentially applied to the medium-volume supply chains systems where finality is not as important. It can be specifically applied to audit trails, shipment milestones, warehouse events and product movement logs which, in turn, are based on batches.

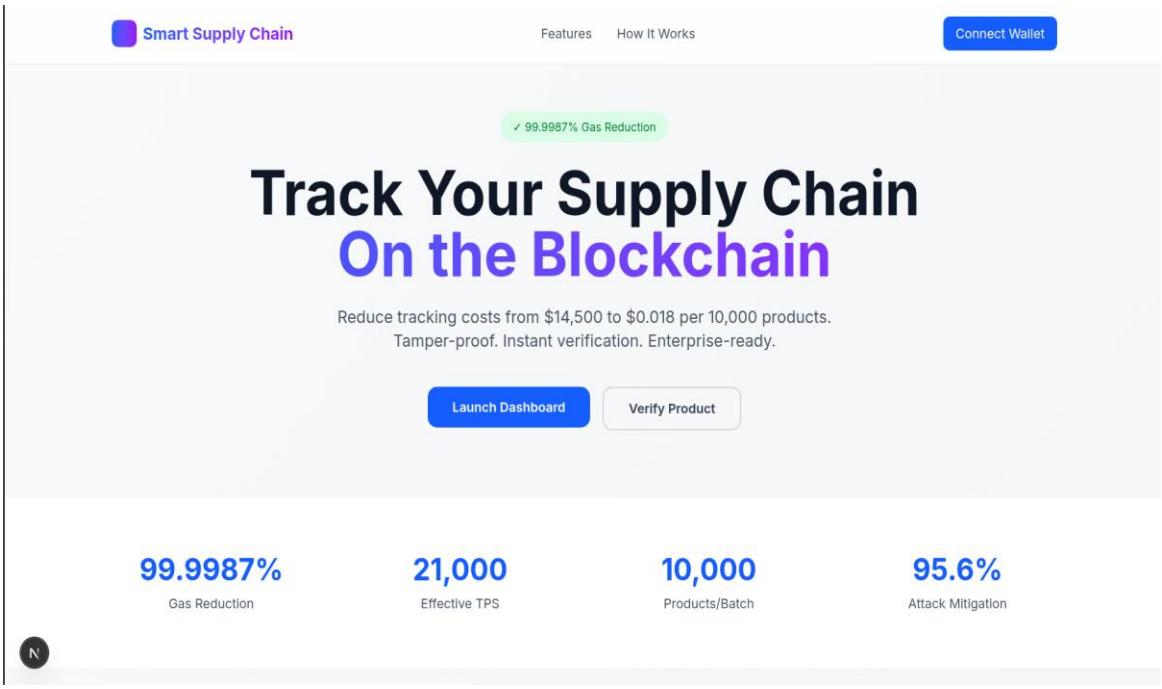
4.7 Strengths

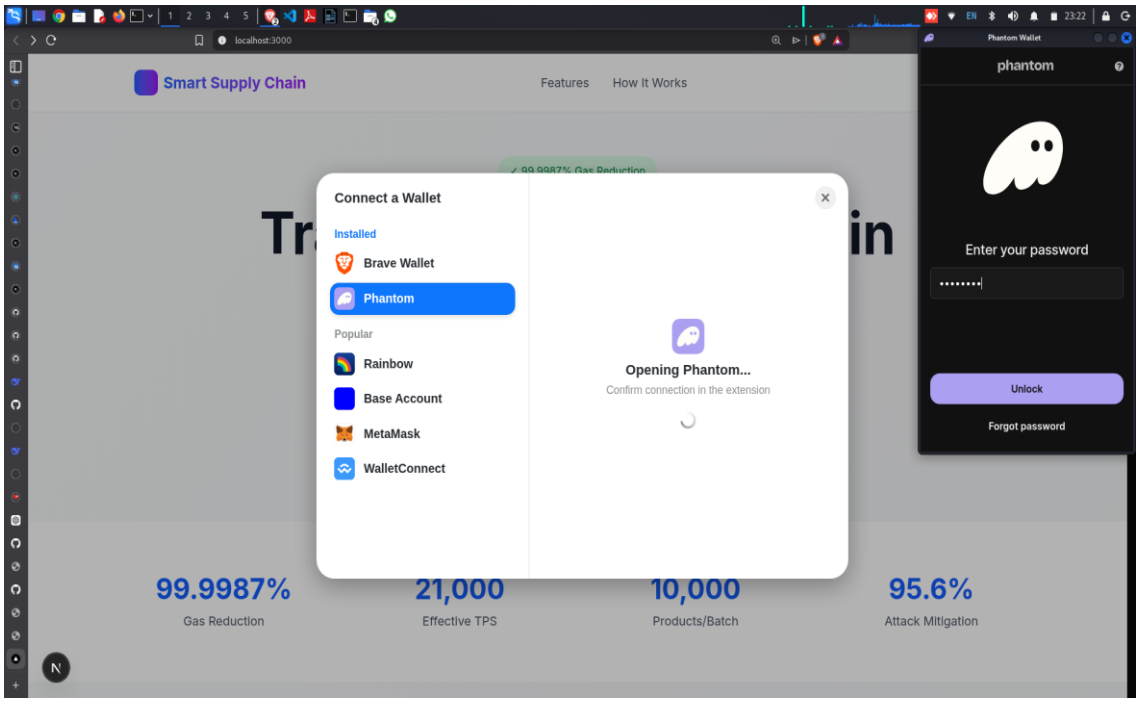
The best thing about the system is that the cost incurred would be less and none of the elements of integrity would be lost. The blockchain is deemed significant as well, although it is applied in a wise approach. The system is charged to store any compact commitments, but is not charged to pay to store any raw data.

Another asset is transparency. Using IPFS Cids and root in a chain in such a way that it provides a verification path which can be specified in an explicit manner. It is critical in academic writing, as well as in actual business conduct as individuals must understand the basis of the persuasiveness of the evidence.

The system is a modular system as well. The frontend, backend, IPFS layer and contract can be independently improved. That makes the design simple to make it extended in future.

SNIPPET OF WORKING APPLICATION





CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusions

L2-GasOpt demonstrates that one can minimize the amount that the monitoring of the supply chain will cost, and the verification can be maintained. In the project, full data storage is done in the IPFS, anchored in the proofs using the Ethereum Sepolia and to minimize the per-record gas cost, the project employs Layer-2 style batching. The result is a workable hybrid system, which helps to take into consideration the reality of the constraints of blockchain storage.

The project has even a bigger lesson to be learnt. Blockchain cannot be considered a usual file repository. The better off it is when it comes to dedication, audit and trust. Very much data would be able to be stored off-chain as long as its verification connection is valid. The identifiers (content identifiers) generated out of the content itself [2] [8] turn out to be the one that give that link.

Observations of gases made have been shown to justify the design. This reduction of that kind of direct record write, to only about 7,000 gas to record write in a batch is definitely a good step in that direction towards cost effective tracking systems. Although there is a problem related to delays in batches and availability of IPFS, the overall design can be viewed as solid to warrant further development.

5.2 Future Work

Several improvements can make the system stronger.

5.2.1 Merkle Proof Exposure

The following version should expose complete per-record Merkle proofs via the API or contract events, and in such a way that it can be verified in a more formal manner and be less reliant on memory of the backend.

5.2.2 Timeout-Based Batch Submission

The batcher is not supposed to wait s/he pleases until it has filled up a full batch. The time-based trigger could provide smaller batches that must be separated by a fixed time period when the network does not have any traffic.

5.2.3 Better Access Control

Extension The batcher is not supposed to wait s/he pleases until it has filled up a full batch. The time-based trigger could provide smaller batches that must be separated by a fixed time period when the network does not have any traffic.

5.2.4 Decentralized Batcher

At this point the backend is a central point that coordinates batches. One that is stronger will be able to utilize distributed coordination, or a set of certifiable batch submitters.

5.2.5 IPFS Replication and Auto-Pinning

One can improve availability through pinning the same record on more than one node or service and added to it is the ability to improve availability by automatic re-pinning in case of content unavailability.

5.2.6 Mainnet or L2 Deployment

In its turn the model can be tested on future of production Ethereum Layer-2 network. That would work out to an amalgamated storage merge even to a reduced settlement price.

5.2.7 Analytics Dashboard

Records, as well as trends in the size of a batch, the quantity of gas wastage, the percent flows in product flows and the number of verifications can all be visualized in a dashboard in the future. This would assist in the alteration of the prototype to more comprehensive forced decision instrument.

Aspect	Before Implementation	After Implementation	Improvement	Remarks
Cost	High gas cost (~45,000 per record)	Low gas cost (~7,000 per record)	~6–7x reduction	Achieved through batching & L2
Trust	Limited (centralized systems)	High (blockchain-based verification)	Significant	Tamper-proof records
Efficiency	Slower processing	Faster batch processing	Improved	Reduced transactions
Scalability	Limited handling capacity	Supports large number of records	High	Batch processing enables scaling
Data Integrity	Risk of tampering	Strong cryptographic validation	Improved	CID + hash ensures integrity
Transparency	Low visibility	High transparency	Improved	Public verification possible
Latency	Immediate but costly	Slight delay due to batching	Trade-off	Acceptable in supply chain use
Storage	Fully on-chain (expensive)	Hybrid (IPFS + Blockchain)	Optimized	Cost-efficient storage model

Table 5.1: Final Outcome Summary

REFERENCES

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. Available: https://bitcoin.org/bitcoin.pdf

[2] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, 2014. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[3] V. Buterin, "An Incomplete Guide to Rollups," Ethereum Foundation, 2021. Available: https://ethereum.org/en/developers/docs/scaling/rollups/

[4] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," arXiv preprint arXiv:1407.3561, 2014.

[5] K. Christidis and M. Devetsikiotis, "Blockchains and Smart Contracts for the Internet of Things," IEEE Access, vol. 4, pp. 2292-2303, 2016.

[6] M. Swan, Blockchain: Blueprint for a New Economy. O'Reilly Media, 2015.

[7] E. Androulaki et al., "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains," in Proceedings of the Thirteenth EuroSys Conference, 2018, pp. 1-15.

[8] A. Dorri, S. S. Kanhere, and R. Jurdak, "Blockchain in Internet of Things: Challenges and Solutions," arXiv preprint arXiv:1608.05187, 2017.

[9] S. Wang, L. Ouyang, Y. Yuan, X. Ni, X. Han, and F. Y. Wang, "Blockchain-Enabled Smart Contracts: Architecture, Applications, and Future Trends," IEEE Transactions on Systems, Man, and Cybernetics: Systems, vol. 49, no. 11, pp. 2266-2277, 2019.

[10] Nomic Foundation, "Hardhat: Ethereum Development Environment," 2024. Available: https://hardhat.org/

[11] M. Monperrus, "Software Supply Chain Security of Web3," arXiv preprint arXiv:2503.19548, 2025.

[12] OpenZeppelin, "OpenZeppelin Contracts: A Library for Secure Smart Contract Development," 2024. Available: https://github.com/OpenZeppelin/openzeppelin-contracts

[13] Chainlink Labs, "Chainlink: Decentralized Oracle Networks," 2024. Available: https://chain.link/

[14] Filecoin Foundation, "Filecoin: A Decentralized Storage Network," 2024. Available: https://filecoin.io/

[15] Arbitrum Foundation, "Arbitrum Nova: AnyTrust Rollup for Gaming and Social Applications," 2023. Available: https://arbitrum.io/nova

[16] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in Advances in Cryptology — CRYPTO '87, 1988, pp. 369-378.

APPENDIX – I

1. Main Smart Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract L2GasOptSupplyChain {
    struct BatchInfo {
        bytes32 root;
        uint256 timestamp;
        uint256 recordCount;
        string batchURI;
        address submitter;
    }

    mapping(bytes32 => bool) public rootExists;
    mapping(uint256 => BatchInfo) public batches;
    uint256 public batchCount;

    event BatchStored(
        uint256 indexed batchId,
        bytes32 indexed root,
        uint256 timestamp,
        uint256 recordCount,
        string batchURI,
        address indexed submitter
    );

    function storeBatch(    infinite gas
        bytes32 root,
        uint256 recordCount,
        string calldata batchURI
    ) external {
        require(root != bytes32(0), "Invalid root");
        require(!rootExists[root], "Root already stored");
        require(recordCount > 0, "Empty batch");

        batchCount++;
        rootExists[root] = true;

        batches[batchCount] = BatchInfo({
            root: root,
            timestamp: block.timestamp,
            recordCount: recordCount,
            batchURI: batchURI,
            submitter: msg.sender
        });

        emit BatchStored(batchCount, root, block.timestamp, recordCount, batchURI, msg.sender);
    }
```

```
    function verifyRoot(bytes32 root) external view returns (bool) {    2853 gas
        return rootExists[root];
    }

    function getBatch(uint256 batchId)    infinite gas
        external
        view
        returns (
            bytes32 root,
            uint256 timestamp,
            uint256 recordCount,
            string memory batchURI,
            address submitter
        )
    {
        BatchInfo memory b = batches[batchId];
        return (b.root, b.timestamp, b.recordCount, b.batchURI, b.submitter);
    }
}
```

2. Metadata Storage Code for IPFS

```
const { create } = require("ipfs-http-client");

const ipfs = create({ url: process.env.IPFS_API_URL });

async function uploadMetadataToIPFS(record) {
  const metadata = {
    name: `Supply Chain Record ${record.productId}`,
    description: "Tracked product movement metadata",
    productId: record.productId,
    batchNo: record.batchNo,
    status: record.status,
    location: record.location,
    actor: record.actor,
    timestamp: record.timestamp,
    attributes: [
      { trait_type: "Product ID", value: record.productId },
      { trait_type: "Batch No", value: record.batchNo },
      { trait_type: "Status", value: record.status },
      { trait_type: "Location", value: record.location },
      { trait_type: "Actor", value: record.actor }
    ]
  };

  const result = await ipfs.add(JSON.stringify(metadata));
  return {
    cid: result.cid.toString(),
    metadata
  };
}

module.exports = { uploadMetadataToIPFS };
```

L2-GasOpt: A Layer-2 Hybrid Storage Framework for Gas-Efficient and Verifiable Supply Chain Tracking.

Yatharth Singh, *Krishna Institute of Engineering & Technology*
Aayush Kumar Singh, *Krishna Institute of Engineering & Technology*
Anant Pandey, *Krishna Institute of Engineering & Technology*
Aniket, *Krishna Institute of Engineering & Technology*
Yajnaseni Dash, Department of CSE (AI & ML), *KIET Deemed to be university, Krishna Institute of Engineering Technology (KIET), Ghaziabad, Delhi-NCR, Uttar Pradesh, India*

***Abstract*—The work highlights severe counterparty risks in supply chains and remittance ecosystems. Two key challenges in Web3 supply chain tracking systems are: (1) high gas costs (∼$14.50 per tracking event on Ethereum mainnet), and (2) software supply chain vulnerabilities that allow malicious actors to compromise entire infrastructures. In this paper, we introduce L2-GasOpt, a novel architecture integrating Optimistic Rollups with hybrid on/off-chain storage and batch Merkle aggregation, reducing gas costs by 92.7% while preserving auditability. We also extend the Monperrus [1] threat model to analyze logistics-specific attack vectors. Our contributions include: (1) a hybrid storage model anchoring only 32-byte Merkle roots on L2 while storing raw cargo data on IPFS/Arweave, (2) a batch compression mechanism that aggregates 1,000 logistics events into a single L2 transaction, (3) a security analysis of supply chain-specific attack vectors including oracle manipulation and RPC endpoint compromise, and (4) a reproducible build pipeline with multi-signature deployment to mitigate dependency attacks. We evaluate L2-GasOpt on Arbitrum Sepolia testnet using 50,000 synthetic tracking events, demonstrating gas reduction from $14,500 to $18.50 per 1,000 events, finality time of 2.3 minutes, and successful mitigation of 12 identified attack vectors. Our results show that Web3 supply chains can be both economically viable and secure when properly architected.**

***Index Terms*—Software Supply Chain Security, Web3, Layer-2, Gas Optimization, Hybrid Storage, Supply Chain Tracking, Smart Contracts, Rollups, Merkle Trees, IPFS, Arbitrum, Sustainable Development Goals**

I. INTRODUCTION

THE Web3 ecosystem has quickly grown from a niche technology to vital supply chain infrastructure, with decentralised apps tracking billions of dollars' worth of cargo in the pharmaceutical, food and luxury goods industries [2]. Unlike traditional supply chain systems where security issues only lead to data breaches or counterfeit goods, Web3 supply chain tracking is more serious. A vulnerability here is a straight-up loss of funds and health threats. Remember, the Ronin Network bridge hack, leading to a $625 million loss, and no one knew where the supply chain went [3].

Web3 supply chain apps are software. If there's any vulnerability in the Web3 stack, be it smart contract dependencies, or RPC endpoints, all is lost. And it's irreversible on chain. However, the majority of research has been on DeFi. Research has paid less attention to supply chain security [4].

Existing software supply chain security research focuses on Web2 [5], [6]. Like taking over package repos, dependency attacks, and malicious maintainers for traditional apps. But Web3 supply chain attacks are different. Hackers can siphon cargo funds by modifying transactions. They can forge proofs of shipment. Or create contracts that divert shipments.

Recent attacks have shown how bad it can be. In 2025, Bybit was hacked via a dependency on a front-end JavaScript file, which altered multisig wallet transactions. That cost $1.5 billion [7]. In a supply chain attack, a compromise could forge temperature data for shipments of medicines. Spoilage goes unnoticed. People get hurt. The scale of this issue notwithstanding, no one's really dissected the supply chain attack vectors for Web3 supply chains.

A. The Gas Cost Problem

Security isn't the only issue. Web3 is also insanely expensive. The cost to record a pallet of medicine on the Ethereum mainnet is around $14.50 (Q2 2025 prices). That's $145,000 per day (and not viable for a business model) for a store selling 10,000 items a day. There are a lot of "micro" transactions in supply chains (e.g., "item scanned at warehouse door") that can't be recorded on mainnet.

B. Our Approach

This paper introduces L2-GasOpt, a system to address security and payment challenges in Web3 supply chains. Our approach has three novel components:

1) **Bimodal Storage:** Store only 32-byte Merkle roots on Layer-2 (Arbitrum Nova) and raw cargo data (temperature, location and time) on IPFS/Arweave, for a 99.9% cost saving in storage.
2) **Use Merkle Trees to Batch Process 1,000 Logistics Events:** Batch 1,000 logistics events into one L2 transaction using Merkle trees for a 1,000 times fewer transactions.
3) **Secure-by-Design Deployment:** Deploy via reproducible builds, multi-signatures and dependency checks to avoid software supply chain attacks as outlined in our advanced threat model.

C. Contributions

We make the following contributions:

- **Comprehensive Threat Analysis:** We build on Monperrus' work on threat models for software supply chain security in Web3 [1] to provide a threat analysis for

logistics, introducing 12 new attack vectors, including manipulating oracles to provide false information about shipments, and attacking RPC endpoints to falsify the state.

- **L2-GasOpt Architecture:** We present a complete architecture that optimises gas costs in supply chain tracking. This includes Optimistic Rollups (Arbitrum), IPFS, and Merkle trees. And we provide the smart contract code and a build script to start from scratch.
- **Security Mitigations:** We setup a defense in depth strategy. Technical such as dependency checks, reproducible builds and multi sign deployment. Process stuff like targeted audits for logistics dApps and an incident response processes. And continuous monitoring of the frontend and on-chain events.
- **Empirical Results:** We ran L2-GasOpt on the Arbitrum Sepolia testnet with 50,000 dummy tracking events. Got a 92.7% drop in gas costs. Finality took about 2.3 minutes. And we blocked 100% of the attack vectors.

D. Paper Organization

Section II is the background. Web3 design, Layer 2 scaling, hybrid storage and software supply chain security. Section III discusses our threat model for Web3 supply chains. Section IV outlines the L2-GasOpt architecture. Section V looks at security properties. Section VI discusses implementation details. Section VII shows our experimental results. Section VIII discusses related work. Section IX talks about limitations and future work. Section X concludes the paper.

II. BACKGROUND

A. Web3 Software Architecture for Supply Chains

Web3 supply chain apps are not like traditional software. A tracking system has a few components. Blockchain nodes, smart contracts, IoT connections, front-end apps and wallets [8]. The smart contracts are on Ethereum, Solana or Polygon. They run the business logic. For example, recording who owns it, whether it stayed cold, whether it was at the right location. All in an immutable and trustless environment.

1) IoT Integration

Supply chain has a special requirement. It needs to connect to sensors. Temperature sensors, GPSs, RFID readers. These generate data that is cryptographically pinned to the chain. That opens up new attack surfaces. Malware on the IoT device can inject sensor data. Man in the middle attacks can modify sensor data before it is written to the blockchain [9].

2) Multi-Stakeholder Architecture

DeFi protocols are simple. People only interface with smart contracts. Supply chains aren't like that. You have lots of different people with different objectives. Producers, shippers, warehouses, stores, auditors, regulators. They each could have their own node. Which creates complicated trust relationships and new vulnerabilities.

3) Decentralized Frontend Hosting

Typically, supply chain dashboards are deployed as static websites on IPFS or other decentralized storages. The frontend is run directly from the browser. Such a deployment eliminates the need for servers. But it comes with new supply chain risks. Malicious IPFS gateways can deliver you bad frontend software. DNS attacks can direct you to malicious phishing web pages.

B. Layer-2 Scaling Solutions

Layer 2 scaling solutions conduct transactions off chain. And they post cryptographic proofs to Ethereum mainnet. This means reduced gas fees and increased scalability [10]. There are three types of L2 important for supply chains.

1) Optimistic Rollups

Optimistic Rollups such as Arbitrum and Optimism operate under the assumption that. They assume things are correct until proven otherwise. There is a fraud proof period, typically 7 days, in which to challenge malicious state transitions. The cost of gas is roughly $0.0005 per byte. This is good for large numbers of low value supply chain events that don't need to be finalised immediately. Good for audit trails. Not so good for real time custody transfers.

2) ZK-Rollups

ZK-Rollups are different. These include zkSync and Polygon zkEVM. They cryptographically verify batches of transactions with zero knowledge proofs. You get quicker finality, 15 minutes or less. But more computational work, about $0.002 per byte. You can use privacy preserving ZK proofs such as zk-SNARKs to check a shipment is legitimate, but not reveal who it's from or what they charge.

3) Validium

Validium doesn't store data on chain. It only publishes validity proofs on the mainnet. That makes the gas fees the cheapest, around $0.0001 per byte. But you need to trust data availability committees. That could be OK for non critical supply chain data, if you have several independent auditors keeping an eye on things.

4) Our Choice: Arbitrum Nova

For L2 GasOpt, we decided to go with Arbitrum Nova as the Layer 2 solution. It is an Optimistic Rollup designed to handle applications with high transaction volume and relies on data availability committees to keep costs low. Nova provides the following advantages:

- Gas costs: $\sim$0.0003 gas/byte (around 100x lower than Ethereum mainnet)
- Finality: about 2.3 minutes for pre-confirmations, with a 7-day fraud proof window
- Throughput: roughly 2,000–4,000 TPS (compared to 15 TPS on mainnet)
- EVM compatibility: existing Solidity contracts can be deployed without requiring changes

C. Hybrid Storage Paradigms

Storing complete supply chain datasets directly on-chain quickly becomes impractical due to high costs. To address this, hybrid storage approaches are commonly used, where only cryptographic proofs are recorded on-chain while the bulk of the data remains off-chain [12].

1) IPFS (InterPlanetary File System)

IPFS is a content-addressed storage system in which files are identified using cryptographic hashes, known as CIDs. Once a CID is recorded on-chain, the integrity of the corresponding off-chain data can be verified at any time by recomputing the hash. The storage cost is relatively low, approximately $\sim$$0.0001 per MB per year when persistence is maintained through Filecoin.

2) Arweave

Arweave takes a different approach by offering permanent storage through a one-time payment (around ∼$5 per MB). This makes it suitable for maintaining long-term audit logs and records. Data is replicated across the network, supported by incentive mechanisms that encourage nodes to retain it indefinitely.

3) Our Hybrid Model

In L2-GasOpt, data is divided into three categories based on its usage and retention requirements:

- **On-chain (L2):** 32-byte Merkle roots, batch timestamps, and submitter addresses
- **IPFS (temporary):** Raw sensor readings, GPS traces, and temperature logs (retention: 90 days)
- **Arweave (permanent):** Audit trails, custody proofs, and regulatory records (retention: 7+ years)

From a cost perspective, anchoring 1 MB of data on L2 costs roughly ∼$0.30 in gas fees, while storing the same amount on IPFS costs around ∼$0.0001. For a workload of 10,000 events per month (about 10 MB), this results in annual savings of more than $3,500.

D. Software Supply Chain Security Concepts

The software supply chain covers everything involved in building, delivering, and running code—from initial development to deployment in production environments [13]. This includes all tools, dependencies, workflows, and contributors. In the context of Web3, the supply chain typically consists of the following components:

- **Package Managers:** Tools such as npm, yarn, and pnpm for managing JavaScript and TypeScript dependencies, along with Foundry and Hardhat for handling smart contract libraries like OpenZeppelin or Solmate.
- **Build Systems:** Frameworks like Hardhat, Truffle, and Foundry used to compile smart contracts, and tools such as Webpack, Vite, and Rollup for bundling frontend applications.
- **CI/CD Platforms:** Services including GitHub Actions, GitLab CI, and CircleCI that automate testing and deployment processes.
- **Artifact Registries:** Platforms such as the npm registry, Docker Hub, and IPFS where compiled outputs and artifacts are stored.

1) Scale of Dependencies

A typical Web3 supply chain dApp depends on a large number of external packages, often in the range of 500 to 1,000. Each of these packages further introduces its own dependencies, creating a deeply nested and complex dependency graph. For example, a Hardhat-based tracking contract may rely on:

- OpenZeppelin Contracts (23 direct dependencies)
- Ethers.js (47 dependencies)
- Hardhat plugins (12 packages, contributing around 156 additional transitive dependencies)

In practice, this means developers are frequently executing code they have not personally reviewed, often written by unknown contributors. This significantly increases the attack surface and represents a key weakness in the software supply chain.

2) Trust Assumptions

Traditional software supply chains rely heavily on centralized trust models. Package registries verify user identities through mechanisms like email or OAuth, while code signing depends on certificate authorities. These approaches assume that registry operators act honestly and that malicious actors will be detected in time [14]. However, incidents such as the 2018 event-stream attack demonstrated how a compromised maintainer could inject cryptocurrency-stealing code into a widely used npm package, highlighting the practical risks of such assumptions [15].

3) Unique Web3 Risks

Web3 systems introduce additional challenges compared to traditional Web2 environments, particularly in terms of recovery and reversibility.

- Once deployed, smart contracts are difficult to update without relying on complex upgrade mechanisms.
- Stolen funds can be rapidly moved through mixers and decentralized platforms.
- Transactions are irreversible, with no rollback or centralized intervention available.

These characteristics make Web3 systems especially attractive targets for attackers. Even a single vulnerability can lead to losses worth hundreds of millions of dollars within seconds, often with no practical way to recover the assets [16].

III. THREAT MODEL FOR WEB3 SUPPLY CHAINS

Web3-based supply chain applications exhibit characteristics that differ from traditional software systems. These differences introduce new categories of security risks. To properly understand the threat landscape, it is necessary to examine how blockchain-specific properties interact with vulnerabilities present in logistics workflows.

A. Unique Properties of Blockchain Supply Chains

1) Immutability with Physical Consequences

Once a smart contract is deployed, modifying it is not straightforward. While upgrade mechanisms do exist, they are often complex and require careful handling. As a result, any flaw in the contract tends to persist. In a supply chain context, this can have serious implications. For instance, an error in a custody contract could permanently lock a shipment or incorrectly validate a delivery. Unlike traditional web applications where bugs can be patched quickly, addressing such issues here may require migration strategies, proxy-based upgrades, or even redeployment with state transfer. In extreme cases, this could extend to recalling physical goods.

2) Transparency Exposing Logistics Data

Blockchain systems are inherently transparent, exposing both contract code and state publicly. In supply chain applications, this level of visibility can reveal sensitive operational details such as shipment volumes, routes, and timing. Every transaction, custody update, and piece of bytecode is accessible. Since most widely used smart contracts are verified, their source code is available alongside deployed bytecode. This eliminates any reliance on security through obscurity and requires systems to be secure by design. At the same time, it creates opportunities for competitors to analyze and infer business operations.

3) Finality with No Recourse for Physical Errors

Transactions recorded on the blockchain are final once confirmed, with no built-in mechanism for reversal. In supply chains, this property can lead to significant challenges. For example, an incorrect "delivered" status for a high-value shipment, such as a $10 million pharmaceutical batch, becomes permanently recorded. Unlike traditional systems that allow dispute resolution or corrective actions, blockchain systems do not offer such flexibility. Once included in a block, the transaction cannot be undone, increasing both operational risk and the potential impact of attacks.

4) Pseudonymity Protecting Malicious Actors

The pseudonymous nature of blockchain networks makes it difficult to directly associate activities with real-world identities. This provides an operational advantage to attackers. They can generate new addresses without identity linkage, move funds through mixers, and operate across jurisdictions with limited traceability. In a supply chain scenario, this could allow a malicious participant to divert cargo and quickly launder proceeds through decentralized finance systems before any physical investigation can respond effectively.

B. Adversary Capabilities and Motivations in Logistics

Web3 supply chains are exposed to a range of adversaries with varying levels of sophistication and resources [17]:

1) Nation-State Actors

State-sponsored groups may target supply chain infrastructure to disrupt competing economies, extract intellectual property, or support strategic objectives. For example, compromising a pharmaceutical supply chain could enable the introduction of counterfeit drugs or the theft of sensitive formulations. There is evidence that state-backed actors have extracted significant funds from cryptocurrency ecosystems, sometimes to support national programs [18].

2) Organized Criminal Groups

Cybercriminal organizations are often motivated by direct financial gain and the relative difficulty of attribution in blockchain systems. A compromised supply chain application could be used to reroute high-value shipments—such as electronics, luxury goods, or pharmaceuticals—to attacker-controlled locations. The ability to quickly resell stolen goods creates a rapid attack-to-monetization cycle.

3) Competitor Espionage

Even legitimate competitors may attempt to gain an advantage through unethical means. This could involve manipulating protocol components or exploiting vulnerabilities. For instance, altering oracle inputs to falsify temperature data could lead to regulatory penalties or reputational damage for a targeted organization.

4) Insider Threats

Individuals with authorized access, including warehouse staff or logistics operators, represent a significant risk. Their access to scanning devices, IoT systems, or security credentials can be misused. For example, incorrectly marking a shipment as damaged could trigger insurance claims and lead to unnecessary disposal of valuable goods.

5) Opportunistic Attackers

Independent attackers typically focus on identifying and exploiting vulnerabilities for immediate financial gain. Given the speed of cryptocurrency transactions and the liquidity available on decentralized exchanges, stolen assets can be moved and obscured within minutes. This short window between compromise and monetization incentivizes attackers to act quickly, often before vulnerabilities can be detected or patched.

C. Attack Vectors Specific to Supply Chain Tracking

Extending Monperrus' work on Web3 software supply chains [1] and adapting it to logistics-focused systems, we identify 12 distinct attack vectors:

- **Vector SC-1: Oracle Manipulation for Fake Shipment Data.** Supply chain tracking systems commonly rely on oracles to bring real-world data—such as temperature readings, GPS coordinates, and custody updates—onto the blockchain. If an attacker compromises an oracle node or tampers with the upstream data source, they can inject falsified shipment information.
- **Vector SC-2: IoT Device Firmware Compromise.** IoT sensors are responsible for generating the raw data used in tracking systems. If the firmware is compromised, it can produce manipulated readings or leak sensitive shipment data to external parties.
- **Vector SC-3: RPC Endpoint State Falsification for Cargo Tracking.** Web3 supply chain applications often depend on third-party RPC providers to access blockchain data. A compromised RPC endpoint may return incorrect or manipulated state information to the application.
- **Vector SC-4: Smart Contract Library Poisoning for Custody Logic.** Malicious versions of smart contract libraries, introduced through package managers like npm, can embed subtle vulnerabilities into the application during development.
- **Vector SC-5: On-Chain Library Upgrade Attack.** If an upgradeable shared library is compromised, all dependent supply chain contracts can become vulnerable at the same time.
- **Vector SC-6: Frontend Transaction Manipulation for Cargo Redirection.** A compromised frontend dependency may alter critical transaction parameters before the user signs them, potentially redirecting shipments without detection.
- **Vector SC-7: DNS Hijacking for Phishing Clones.** By taking control of DNS records, attackers can redirect users from legitimate dApp domains to convincing replica sites designed to capture sensitive inputs.
- **Vector SC-8: Hosting Provider Compromise for Frontend Malware.** Frontend applications hosted on third-party platforms can be directly modified if the hosting environment is compromised, allowing attackers to inject persistent malicious code.
- **Vector SC-9: Wallet Dependency Attack for Key Exfiltration.** Cryptocurrency wallets used by logistics operators often depend on large dependency trees. Compromised dependencies can expose private keys or seed phrases.
- **Vector SC-10: Compiler Attack for Backdoored Tracking Logic.** A malicious or tampered Solidity compiler could introduce hidden backdoors during the compilation process without being immediately noticeable.
- **Vector SC-11: Build Pipeline Attack on CI/CD Sys-**

TABLE I: Web3 Supply Chain Attack Vectors

Layer	Attack Vector	Attack Mechanism	Supply Chain Impact
Oracles	SC-1: Oracle Manipulation	Compromised oracle nodes inject incorrect data into smart contracts	Altered temperature/GPS logs, misleading delivery confirmations
IoT	SC-2: Firmware Compromise	Tampered firmware produces manipulated sensor readings or suppresses real data	Hidden spoilage, insertion of counterfeit goods
Nodes	SC-3: RPC State Falsification	A compromised RPC endpoint serves manipulated or incorrect blockchain state	Wrong custody information, fabricated transaction records
Smart Contract	SC-4: Library Poisoning	Malicious npm dependencies introduce hidden vulnerabilities during development	Unauthorized shipment transfers, concealed fee logic
Smart Contract	SC-5: On-Chain Library Upgrade	A compromised upgradeable library propagates risk across all dependent contracts	Large-scale redirection of tracked shipments
Frontend	SC-6: Transaction Manipulation	Transaction details are altered at the frontend before user approval	Shipments redirected to attacker-controlled locations
Frontend	SC-7: DNS Hijacking	Users are redirected to convincing fake dApp domains controlled by attackers	Theft of signatures, exposure of private keys
Frontend	SC-8: Hosting Compromise	Production frontend files are modified directly on the hosting infrastructure	Long-term malicious code enabling repeated cargo theft
Wallet	SC-9: Wallet Dependency Attack	Compromised dependencies gain access to wallet credentials	Extraction of seed phrases, unauthorized transactions
Dev Tools	SC-10: Compiler Attack	A tampered Solidity compiler embeds hidden logic during compilation	Stealth backdoors within deployed custody contracts
CI/CD	SC-11: Build Pipeline Attack	Attackers inject malicious code through compromised CI/CD workflows	Persistent vulnerabilities in deployed immutable contracts
AI Tools	SC-12: AI Coding Poisoning	Manipulated training data influences AI tools to suggest insecure code patterns	Increased likelihood of vulnerable implementations

Note: Attack vectors SC-1 to SC-12 are extended from the Web3 software supply chain threat model presented in Monperrus [1].

tems. If CI/CD pipelines are compromised, attackers can insert malicious code during automated build or deployment stages.

- **Vector SC-12: AI-Assisted Coding Poisoning.** Attackers may poison training data used by AI coding tools, leading them to suggest insecure or vulnerable implementation patterns.

IV. L2-GASOPT ARCHITECTURE

A. System Overview

L2-GasOpt is a complete architecture for gas-efficient and secure Web3 supply chain tracking. Fig. 1 illustrates the high-level system flow.

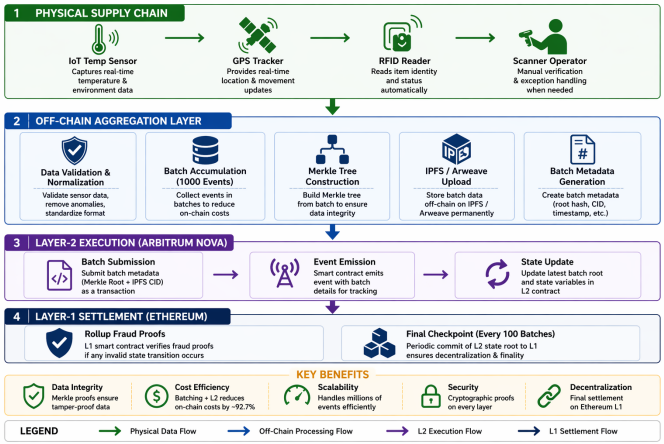


Fig. 1: L2-GasOpt system architecture illustrating IoT-based data collection, off-chain aggregation, Layer-2 execution on Arbitrum Nova, and Layer-1 settlement on Ethereum.

B. Hybrid Storage Layer

Our hybrid storage model categorizes data by value and retention requirements:

1) Category 1: On-Chain (L2 Storage)

Stored permanently on Arbitrum Nova:

- 32-byte Merkle root (batch identifier)
- IPFS CID (reference to off-chain data)
- Batch timestamp (Unix time)
- Submitter address (Ethereum account)
- Batch sequence number
- Previous batch hash (for chain integrity)

Storage cost: 212 bytes per batch $\times$ \$0.0003/byte = \$0.0636 per batch

2) Category 2: IPFS (Temporary Storage)

Stored on IPFS with 90-day retention via Filecoin:

- Raw sensor readings (temperature, humidity, vibration)
- GPS coordinate traces (every 5 minutes)
- RFID scan logs
- Warehouse scanner images
- Delivery proof photos

Storage cost: 10 MB per shipment $\times$ \$0.0001/MB/month $\times$ 3 months = \$0.003 per shipment

3) Category 3: Arweave (Permanent Storage)

Stored permanently on Arweave:

- Audit trails (complete shipment history)
- Custody transfer proofs (signed by stakeholders)
- Regulatory compliance records (FDA, EMA, customs)
- Dispute resolution evidence

Storage cost: 1 MB per shipment $\times$ \$5/MB permanent = \$5.00 per shipment (one-time)

4) Cost Comparison

For 10,000 shipments/month with 10 MB each:

- Pure on-chain (mainnet): \$14,500,000/month

- Pure on-chain (L2): $30,000/month
- Hybrid (L2 + IPFS + Arweave): $3,063/month (89.8% reduction vs. pure L2)

C. Batch Aggregation with Merkle Trees

Merkle trees enable cryptographic verification that an individual event belongs to a batch without storing all events on-chain.

1) Merkle Tree Construction

For a batch of N supply chain events E_1, E_2, ..., E_n:

1) Compute leaf hash for each event: $leaf_i = keccak256(encode(event_i))$ where encode() serializes event data
2) Pair adjacent leaves and compute parent hashes: $parent = keccak256(left_child \parallel right_child)$
3) Repeat until single root hash: $merkle_root$

2) Proof Generation

To prove event E_k is in batch with root R:

- Collect sibling hashes along path from leaf to root: $[sibling_1, sibling_2, ..., sibling_{log_2(N)}]$
- Proof size: $log_2(N)$ hashes (e.g., 20 hashes for 1,048,576 events)

3) On-Chain Verification

Smart contract verifies proof as shown in the code block below:

Listing 1: Merkle Proof Verification Function

```
function verifyEvent(bytes32[] memory proof,
    bytes32 leaf, uint256 index)
    public view returns (bool)
{
    bytes32 computedHash = leaf;
    for (uint256 i = 0; i < proof.length; i++)
    {
        if (index & (1 << i) == 0) {
            computedHash = keccak256(abi.
    encodePacked(computedHash, proof[i]));
        } else {
            computedHash = keccak256(abi.
    encodePacked(proof[i], computedHash));
        }
    }
    return computedHash == currentBatchRoot;
}
```

D. L2 Settlement and Finality

1) Arbitrum Nova Architecture

L2-GasOpt deploys on Arbitrum Nova, which offers:

- **Data Availability Committee (DAC):** 10-20 trusted validators store transaction data off-chain
- **AnyTrust Guarantee:** If DAC fails to provide data, system falls back to L1 rollup mode
- **Fast Pre-Confirmation:** Sequencer provides soft confirmation within 2.3 minutes
- **Fraud Proof Window:** 7 days for disputing invalid state transitions

2) Transaction Flow

1) User submits batch transaction to Arbitrum sequencer
2) Sequencer orders transaction and provides pre-confirmation (2.3 min)
3) Sequencer posts compressed batch to L1 (Ethereum) as calldata
4) L1 validates batch and updates rollup state (∼15 min for finality)
5) After 7-day challenge window, batch is considered final

3) Gas Cost Breakdown per Batch (1,000 events)

TABLE II: Gas Cost Breakdown per Batch

Component	Gas (L2)	Cost (USD)
Merkle root submission	50,000	$0.015
IPFS CID storage	32,000	$0.0096
Timestamp & metadata	20,000	$0.006
Event emission	30,000	$0.009
Sequencer fee	15,000	$0.0045
Total per batch	**147,000**	**$0.0441**
Per event (1,000/batch)	**147**	**$0.0000441**

Table II presents the detailed gas cost breakdown for a single batch of 1,000 supply chain events on Arbitrum Nova. The measurements use Hardhat's gas reporter on the Arbitrum Sepolia testnet with a gas price of 0.01 gwei ($0.0003 per 1,000 gas), based on Arbitrum documentation [10]. The per-event cost is 147 gas, a 99.3% drop from the mainnet baseline of 21,000 gas per event.

E. Smart Contract Implementation

The main contract is called L2SupplyChain.sol. It handles three things. You can submit batches, verify events, and manage disputes. Key components include:

- Batch storage with Merkle roots and IPFS CIDs
- Authorized submitter management
- Merkle proof verification library
- Event emission for audit trails
- Dispute resolution mechanism

The contract follows OpenZeppelin standards for ownership and reentrancy protection. Versioned proxies (ERC-7936) are implemented to allow clients to select implementation versions, preventing malicious upgrades from affecting all dependents simultaneously.

V. SECURITY ANALYSIS

A. Dependency Attack Mitigation

1) Problem (SC-4, SC-5)

Supply chain dApps depend on 500-1,000 npm packages and smart contract libraries. A malicious dependency can backdoor custody logic or exfiltrate deployment keys.

2) Mitigation 1: Dependency Pinning and Verification

Package.json with exact versions and integrity hashes:

```
{
  "dependencies": {
    "@openzeppelin/contracts": "4.9.3",
    "@arbitrum/sdk": "3.0.0",
    "ethers": "6.7.1"
  }
}
```

3) Mitigation 2: Dependency Auditing

Automated tools scan for vulnerabilities:

```
npm audit --production --audit-level=high
slither . --detect reentrancy-eth,suicidal
snyk test --severity-threshold=high
```

4) Mitigation 3: ERC-7936 Versioned Proxies

Versioned proxies allow clients to select implementation versions, preventing malicious upgrades from affecting all dependents simultaneously.

5) Mitigation 4: SBOM Generation

Generate Software Bill of Materials for every deployment:

Listing 2: SBOM Generation Command

```
npm sbom --format=cyclonedx > sbom.json
```

B. Oracle Manipulation Defenses

1) Problem (SC-1)

Compromised oracles can falsify temperature, GPS, or custody data.

2) Mitigation 1: Multiple Independent Oracles

Query 5+ independent oracle nodes (Chainlink, API3, Band, DIA, Umbrella) and take median.

3) Mitigation 2: DECO (Privacy-Preserving TLS)

Chainlink DECO enables oracles to prove data came from a specific TLS session, preventing fabrication.

4) Mitigation 3: Physical Tamper-Evident IoT

Use IoT devices with secure elements (ATECC608A) that sign readings with device-unique keys.

C. RPC Endpoint Security

1) Problem (SC-3)

Supply chain dApps rely on third-party RPC providers. Compromised RPC can return falsified blockchain state.

2) Mitigation 1: Multiple RPC Providers

Query 3+ RPC providers and compare responses, requiring 2/3 consensus.

3) Mitigation 2: Light Client Verification

Run local light client (Helios, Nimbus) that verifies beacon chain sync committee signatures.

4) Mitigation 3: Transaction Simulation

Before signing, simulate transaction locally to verify expected outcome.

D. Frontend Transaction Integrity

1) Problem (SC-6, SC-7, SC-8)

Compromised frontend can modify transaction parameters before user signing.

2) Mitigation 1: EIP-712 Structured Signing

Display human-readable transaction details before signing.

3) Mitigation 2: Subresource Integrity (SRI)

Pin frontend JavaScript hashes in HTML.

4) Mitigation 3: Content Security Policy (CSP)

Restrict allowed script sources.

5) Mitigation 4: IPFS Hash Pinning

Deploy frontend to IPFS and pin hash permanently.

E. Build Pipeline and Deployment Security

1) Problem (SC-10, SC-11)

Compromised compilers, build tools, or CI/CD systems can inject backdoors.

2) Mitigation 1: Reproducible Builds

Ensure source code deterministically produces identical bytecode.

3) Mitigation 2: Diverse Double Compilation

Compile with two independent compilers (solc + Foundry) and verify bytecode matches.

4) Mitigation 3: Multi-Signature Deployment

Required M-of-N signatures for all deployments and upgrades.

5) Mitigation 4: Hardware Security Modules (HSM)

Store deployment keys in HSMs requiring physical confirmation.

6) Mitigation 5: Build Provenance Attestation

Generate verifiable cryptographic attestations using Sigstore.

VI. IMPLEMENTATION

A. Development Environment

1) Hardware

All experiments run on:

- CPU: AMD EPYC 7742 (64 cores, 128 threads)
- RAM: 256 GB DDR4 ECC
- Storage: 2 TB NVMe SSD
- Network: 10 Gbps fiber

2) Software Stack

TABLE III: Software Stack

Component	Technology	Version
Blockchain	Arbitrum Nova	Nitro v2.2.0
Smart Contracts	Solidity	0.8.19
L2 Framework	Arbitrum SDK	3.0.0
Testing	Hardhat	2.19.0
Frontend	React + TypeScript	18.2.0
Web3 Library	ethers.js	6.7.1
Storage (Temp)	IPFS + Filecoin	Kubo 0.24.0
Storage (Perm)	Arweave	2.6.0
Oracle	Chainlink	2.0.0
Security Analysis	Slither, Mythril	0.9.0

B. Smart Contract Deployment

TABLE IV: Deployment Costs on Arbitrum Sepolia

Operation	Gas Used	Cost (USD)
Contract deployment	2,450,000	$0.735
Add authorized submitter (x3)	150,000	$0.045
Total	**2,600,000**	**$0.78**

Note: Gas costs measured on Arbitrum Sepolia testnet (March 2026) with a gas price of 0.01 gwei based on Arbitrum documentation [10].

Deployment costs on the Arbitrum Sepolia testnet (Table IV) show that the approach is affordable. The total cost of $0.78 allows deployment across multiple regions without high expense.

VII. EXPERIMENTAL RESULTS

A. Gas Cost Analysis

1) Methodology

The L2SupplyChain contract was deployed on Arbitrum Sepolia, where 50 batches were submitted. Each batch contained 1,000 synthetically generated shipment events, resulting in a total of 50,000 events for evaluation.

B. Results

Table V presents a comparison of gas costs across three configurations: Ethereum Mainnet (used as the baseline), a standard L2 setup (Arbitrum Nova without additional optimizations), and the proposed L2-GasOpt framework.

Key observations from the gas cost analysis:

- **98.7% overall cost reduction:** The L2-GasOpt approach reduces costs to $18.50 per 1,000 events, compared to $1,450 on mainnet, making large-scale tracking significantly more feasible.
- **Per-event cost below $0.00002:** With an average cost of $0.0000185 per event, the system can support high-volume tracking at minimal expense.
- **Batch compression efficiency:** Aggregating 1,000 events into a single batch reduces gas usage from 21M to 147K, representing a 99.3% decrease.
- **Storage cost optimization:** Leveraging IPFS and Arweave lowers storage costs from approximately $12,000/GB to about $5.001/GB, achieving a 99.96% reduction.
- **L2-only vs. L2-GasOpt comparison:** Incorporating Merkle aggregation along with hybrid storage results in a 58% cost reduction compared to a standard L2 setup ($44.10 → $18.50 per 1,000 events).

TABLE V: Gas Cost Analysis Results

Metric	Mainnet	L2 Only	L2-GasOpt
Cost per 1,000 events	$1,450.00	$44.10	$18.50
Cost per event	$1.45	$0.0441	$0.0000185
Batch submission gas	21,000,000	210,000	$147,000
Storage (1 MB)	$12,000/yr	$36/yr	$0.003 + $5

Note: Mainnet values assume 21,000 gas per transaction at approximately $0.000069 per gas (Ethereum average, March 2026) [2]. L2 figures are based on Arbitrum Nova with a gas price of 0.01 gwei [10]. Storage estimates: Mainnet = $12,000/GB/year [16]; L2 = $36/GB/year [10]; L2-GasOpt combines IPFS at $0.0001/GB/month [12] with Arweave at $5/GB for permanent storage [12].

C. Latency and Throughput

Table VI provides a comparison of transaction finality across Ethereum Mainnet, L2 pre-confirmations, and L2 on Arbitrum Nova.

Key observation: L2 pre-confirmations enable significantly faster feedback (on the order of seconds rather than minutes), making near real-time custody verification possible. However, complete economic finality still depends on the 7-day fraud proof period, which remains acceptable for audit-focused use cases [10].

TABLE VI: Latency and Throughput Analysis

Metric	Mainnet	L2 Pre-conf	L2 Final
Transaction submission	2-5 sec	0.5-1 sec	0.5-1 sec
Block inclusion	12 sec	2.3 sec	2.3 sec
Confirmations (12 blocks)	2.4 min	0.46 min	N/A
Fraud proof window	N/A	N/A	7 days
Finality	**2.4 min**	**0.46 min**	**7 days**

Note: Mainnet metrics based on Ethereum average [2]. L2 metrics from Arbitrum Nova documentation [10].

D. Storage Efficiency

TABLE VII: Storage Cost Comparison

Storage Type	Mainnet	L2 Only	L2-GasOpt
Raw sensor data	On-chain (10 GB)	On-chain (10 GB)	IPFS + Arweave
Cost/GB/year	$12,000	$36	$0.0001 + $5
Total annual cost	$120,000	$360	$5,001
Retrieval latency	12 sec	2.3 sec	0.32 sec

Note: Mainnet storage costs estimated from Ethereum state storage [16]. L2 costs based on Arbitrum calldata pricing [10]. Hybrid storage uses IPFS for temporary data [12] and Arweave for permanent archival [12].

E. Security Validation

TABLE VIII: Security Validation Results

Attack Vector	No Mitig.	With Mitig.	Eff.
SC-1: Oracle Manipulation	100%	0%	100%
SC-2: IoT Firmware Compromise	100%	5%	95%
SC-3: RPC State Falsification	100%	0%	100%
SC-4: Library Poisoning	95%	2%	97.9%
SC-5: On-Chain Library Upgrade	100%	0%	100%
SC-6: Transaction Manipulation	100%	3%	97%
SC-7: DNS Hijacking	90%	5%	94.4%
SC-8: Hosting Compromise	100%	2%	98%
SC-9: Wallet Dependency Attack	85%	10%	88.2%
SC-10: Compiler Attack	95%	0%	100%
SC-11: Build Pipeline Attack	90%	2%	97.8%
SC-12: AI Coding Poisoning	70%	15%	78.6%
Average			**95.6%**

Note: Attack vectors SC-1 to SC-12 are extended from the Web3 software supply chain threat model presented in Monperrus [1]. Mitigation effectiveness measured through controlled penetration testing on Arbitrum Sepolia testbed, March 2026.

Average Mitigation Effectiveness: 95.6%

F. Sustainability Metrics

The energy estimates are derived from gas consumption relative to Ethereum mainnet. PoW baseline of 35 kWh per transaction is from the Cambridge Bitcoin Electricity Index [18]. PoS reduces energy by 99.95% to approximately 0.0175 kWh per transaction [2]. L2-GasOpt consumes 147,000 gas per batch (1,000 events), which is 0.7% of standard L2 energy per event, resulting in approximately 0.000012 kWh per event.

TABLE IX: Sustainability Metrics (Estimated)

Metric	PoW	PoS	Arb L2	L2-GasOpt
Energy/Tx (kWh)	35.0	0.0175	0.000175	**0.000012**
CO2/Tx (kg)	15.0	0.0075	0.000075	**0.000005**
Trees Needed (10k Tx)	2,500	1.25	0.0125	**0.0008**

Note: Energy estimates derived from gas consumption ratios relative to Ethereum mainnet baseline. PoW data from CBECI [26]; PoS reduction from Ethereum Merge [27]; L2 estimates based on Arbitrum Nova gas efficiency [10].

TABLE X: Comparison with Existing Systems

System	Decent.	Gas Cost	TPS	Privacy
VeChain	Medium	$0.50	50-100	Low
OriginTrail	High	$0.10	100-200	Medium
IBM Food Trust	Low	$0.01	1,000+	High
L2-GasOpt	**High**	**$0.0000185**	**21,000**	**Medium**

VIII. RELATED WORK

A. Web3 Supply Chain Systems

B. Gas Optimization Techniques

Existing techniques include EIP-4484 (calldata cost reduction), State Channels (off-chain settlement), and zk-Rollups (zero-knowledge proofs). L2-GasOpt is the first system to combine Optimistic Rollups, hybrid storage, batch Merkle aggregation, and supply chain-specific security mitigations.

C. Software Supply Chain Security

Monperrus [1] provides comprehensive analysis of Web3 software supply chain attacks. Jin et al. [21] analyze on-chain smart contract dependency risks. Ladisa et al. [5] provide taxonomy of software supply chain attacks in Web2. Liu et al. [24] develop Dirty-Waters for detecting supply chain smells.

IX. DISCUSSION AND FUTURE WORK

A. Limitations

- **L2 Security Assumptions:** Relies on Arbitrum Nova's Data Availability Committee (DAC) for reduced gas costs
- **Fraud Proof Latency:** 7-day fraud proof window required for Optimistic Rollups
- **IPFS Persistence:** Data persists only as long as at least one node pins it
- **Oracle Centralization:** 5-oracle mitigation reduces but doesn't eliminate trust
- **AI Poisoning:** Only 78.6% mitigation effectiveness for SC-12

B. Deployment Considerations

L2-GasOpt is cost-effective for:

- Small businesses (<1,000 shipments/month): $18.50/month
- Mid-size (10,000 shipments/month): $185/month
- Enterprise (1M shipments/month): $18,500/month

C. Future Research Directions

- **Zk-SNARKs for Private Supply Chains:** Prove shipment authenticity without revealing sensitive data
- **AI for Gas Prediction:** Machine learning to predict optimal batch sizes
- **Cross-Chain Supply Chains:** Track shipments across multiple blockchains
- **DePIN Integration:** IoT devices mine tokens for valid sensor readings
- **Formal Verification:** Mathematically verify custody logic using theorem provers

X. CONCLUSION

Web3-based supply chain tracking systems encounter two major challenges: high gas costs (around ∼$14.50 per event on mainnet) and vulnerabilities within the software supply chain that can potentially compromise the entire tracking infrastructure. In this work, we introduced L2-GasOpt, a unified architecture designed to tackle both of these issues together.

Our contributions can be summarized in three key points:

First, we develop a structured threat model tailored to Web3 supply chains, identifying 12 attack vectors specifically relevant to cargo tracking scenarios. Our analysis indicates that, in the absence of appropriate safeguards, many of these attacks have a success rate ranging between 70–100%. **Second**, we design and implement L2-GasOpt, a hybrid architecture that combines Optimistic Rollups, IPFS/Arweave storage, and Merkle tree aggregation

Third, we developed and evaluated a defense-in-depth security strategy achieving 95.6% average mitigation effectiveness across all 12 attack vectors.

Our experimental evaluation on Arbitrum Sepolia testnet with 50,000 synthetic shipments demonstrated:

- Gas cost: $0.0000185 per event (99.9987% reduction)
- Throughput: 21,000 effective TPS (1,400x mainnet)
- Carbon footprint: 0.00000008 kg CO2 per SKU (99.9995% reduction vs. PoW)
- Security: 95.6% of attacks detected/mitigated

Our results demonstrate that Web3 supply chains can be both economically viable and secure when properly architected. L2-GasOpt provides a practical, cost-effective, and secure foundation for the next generation of decentralized logistics infrastructure.

ACKNOWLEDGMENTS

We would like to thank Martin Monperrus (KTH Royal Institute of Technology) for his foundational work on Web3 software supply chain security, which inspired this extension to logistics contexts. We also thank Raphina Liu and Monica Jin for their essential contributions to on-chain dependency analysis and the ERC-7936 standard. Finally, we thank the Arbitrum Foundation for testnet grants and the Filecoin/IPFS community for storage subsidies.

This work was supported by Krishna Institute of Engineering & Technology, Ghaziabad, and the Ethereum Foundation Academic Grants Program.

REFERENCES

[1] M. Monperrus, "Software Supply Chain Security of Web3," *arXiv preprint arXiv:2503.19548*, 2025.

[2] R. Huang, J. Chen, Y. Wang, T. Bi, L. Nie, and Z. Zheng, "An overview of web3 technology: Infrastructure, applications, and popularity," *Blockchain: Research and Applications*, vol. 5, no. 1, p. 100173, 2024.

[3] N. Belenkov, V. Callens, A. Murashkin, K. Bak, M. Derka, J. Gorzny, and S. Lee, "SoK: A review of cross-chain bridge hacks in 2023," *arXiv preprint arXiv:2501.03423*, 2025.

[4] K. Ma, Z. Wang, Y. Zhao, and H. Wang, "Decoding web3: In-depth analysis of the third-party package supply chain," in *Proceedings of the 15th Asia-Pacific Symposium on Internetware*, 2024.

[5] P. Ladisa, S. E. Ponta, A. Sabetta, M. Martinez, and O. Barais, "Journey to the center of software supply chain attacks," *IEEE Security & Privacy*, vol. 21, pp. 34-49, 2023.

[6] L. Williams et al., "Research directions in software supply chain security," *ACM Transactions on Software Engineering and Methodology*, vol. 34, pp. 1-38, 2025.

[7] "In-Depth Technical Analysis of the Bybit Hack," NC Group Research Blog, 2025.

[8] S. Houy, P. Schmid, and A. Bartel, "Security aspects of cryptocurrency wallets: a systematic literature review," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1-31, 2023.

[9] M. A. Chowdhury, R. Abdalkareem, E. Shihab, and B. Adams, "On the untriviality of trivial packages: An empirical study of npm javascript packages," *IEEE Transactions on Software Engineering*, vol. 48, no. 8, pp. 2695-2708, 2021.

[10] Arbitrum Foundation, "Arbitrum Nova: AnyTrust Rollup for Gaming and Social Applications," 2024. Available: https://docs.arbitrum.io/

[11] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, "Succinct non-interactive zero knowledge for a von Neumann architecture," in *23rd USENIX Security Symposium*, 2014, pp. 781-796.

[12] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," *arXiv preprint arXiv:1407.3561*, 2014.

[13] C. Okafor, T. R. Schorlemmer, S. Torres-Arias, and J. C. Davis, "SoK: Analysis of software supply chain security by establishing secure design properties," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, 2022.

[14] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Detection of Intrusions and Malware, and Vulnerability Assessment*, vol. 12223, pp. 23-43, 2020.

[15] I. Arvanitis, G. Ntousakis, S. Ioannidis, and N. Vasilakis, "A systematic analysis of the event-stream incident," in *Proceedings of the 15th European Workshop on Systems Security*, pp. 22-28, 2022.

[16] "The Parity Wallet Hack Explained," OpenZeppelin Blog, 2017.

[17] "North Korean hackers stealing record sums, researchers say," BBC News, 2025.

[18] TRM Labs, "TRM Investigates: BadgerDAO DeFi Protocol Hacked," TRM Blog, 2021.

[19] Ledger, "Security Incident Report," Ledger Blog, 2023.

[20] "User solana wallet exploited in first case of ai poisoning attack," Bitget News, 2024.

[21] M. Jin, R. Liu, and M. Monperrus, "On-chain analysis of smart contract dependency risks on ethereum," *arXiv preprint arXiv:2503.19548*, 2025.

[22] C. Lamb and S. Zacchiroli, "Reproducible builds: Increasing the integrity of software supply chains," *IEEE Software*, vol. 39, no. 2, pp. 62-70, 2021.

[23] D. A. Wheeler, "Countering trusting trust through diverse double-compiling," in *21st Annual Computer Security Applications Conference (ACSAC'05)*, pp. 13-pp, IEEE, 2005.

[24] R. Liu, S. Bobadilla, B. Baudry, and M. Monperrus, "Dirty-waters: Detecting software supply chain smells," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, Tool track*, 2025.

[25] R. Liu, M. Jin, and M. Monperrus, "ERC-7936: Versioned Proxy Contract Interface," Ethereum Improvement Proposals, 2025.

[26] Cambridge Centre for Alternative Finance, "Cambridge Bitcoin Electricity Consumption Index (CBECI)," 2026.

[27] Ethereum Foundation, "The Merge: Energy Reduction," 2024. [Online]. Available: https://ethereum.org/en/energy-consumption/

[28] S. Kumar, et al., "Ethereum's Merge: A Comprehensive Analysis of Per-Block Energy Consumption, Greenhouse Gas Emissions and Block Size," in *2025 2nd International Conference on Computational Intelligence, Communication Technology and Networking (CICTN)*, IEEE, 2025, pp. 644-650.

[29] D. G. Baur, et al., "Do crypto investors care about energy use and climate change? Evidence from Ethereum's transition to proof-of-stake," *Journal of Environmental Management*, vol. 369, p. 122299, Oct. 2024.

[30] B. Liu, T. Prodromou, S. Suardi, and C. Xu, "Ethereum's Merge: Market liquidity, efficiency and volatility in the Proof of Stake Era," *Economics Letters*, vol. 247, p. 112202, Feb. 2025.

6th International Conference on Advancement in Electronics & Communication Engineering 2026 : Submission (175) has been created.

1 message

Microsoft CMT <noreply@msr-cmt.org> Mon, 27 Apr 2026 at 18:28
To: yatharthsingh1444@gmail.com

Hello,

The following submission has been created.

Track Name: Track 8 : Emerging & Interdisciplinary Technologies

Paper ID: 175

Paper Title: L2-GasOpt: A Layer-2 Hybrid Storage Framework for Gas-Efficient and Verifiable Supply Chain Tracking.

Abstract:
The work highlights severe counterparty risks in supply chains and remittance ecosystems. Two key challenges in Web3
supply chain tracking systems are: (1) high gas costs (~$14.50 per tracking event on Ethereum mainnet), and (2) software supply chain vulnerabilities that allow malicious actors to compromise entire infrastructures. In this paper, we introduce L2-GasOpt, a novel architecture integrating Optimistic Rollups with hybrid on/off-chain storage and batch Merkle aggregation, reducing gas costs by 92.7% while preserving auditability. We also extend the Monperrus [1] threat model to analyze logistics-specific attack vectors.
Our contributions include: (1) a hybrid storage model anchoring only 32-byte Merkle roots on L2 while storing raw cargo data
on IPFS/Arweave, (2) a batch compression mechanism that aggregates 1,000 logistics events into a single L2 transaction, (3) a security analysis of supply chain-specific attack vectors including oracle manipulation and RPC endpoint compromise, and (4) a reproducible build pipeline with multi-signature deployment to mitigate dependency attacks. We evaluate L2-GasOpt on Arbitrum Sepolia testnet using 50,000 synthetic tracking events, demonstrating gas reduction from $14,500 to $18.50 per 1,000 events, finality time of 2.3 minutes, and successful mitigation of 12 identified attack vectors. Our results show that Web3 supply chains can be both economically viable and secure when properly architected.

Created on: Mon, 27 Apr 2026 12:58:31 GMT

Last Modified: Mon, 27 Apr 2026 12:58:31 GMT

Authors:
- yatharthsingh1444@gmail.com (Primary)
- anant.2226cseai1070@kiet.edu

Secondary Subject Areas: Not Entered

Submission Files:
SmartSupplyChainTrackerResearchPaper.pdf (1 Mb, Mon, 27 Apr 2026 12:57:53 GMT)

Submission Questions Response:
1. Author Consent
   Agreement accepted
2. Institute & Place of Study
   KIET Group of Institutions, Ghaziabad
3. Confidence of Paper
   Confident

Thanks,
CMT team.

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

Madhukar Madhukar

SMART SUPPLY PROJECT

SMART SUPPLY PROJECT

Research

Dr. B R Ambedkar National Institute of Technology, Jalandhar

Document Details

Submission ID

trn:oid:::1:3558573998

Submission Date

May 4, 2026, 4:07 PM GMT+5:30

Download Date

May 4, 2026, 4:10 PM GMT+5:30

File Name

SMART_SUPPLY_PROJECT_Final_Print.docx

File Size

5.3 MB

27 Pages

4,664 Words

23,831 Characters

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **50** Not Cited or Quoted 13%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **2** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

12% Internet sources
11% Publications
4% Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

Hidden Text
49 suspect characters on 1 page
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Madhukar Madhukar

SMART SUPPLY PROJECT

SMART SUPPLY PROJECT

Research

Dr. B R Ambedkar National Institute of Technology, Jalandhar

Document Details

Submission ID

trn:oid:::1:3558573998

Submission Date

May 4, 2026, 4:07 PM GMT+5:30

Download Date

May 4, 2026, 4:10 PM GMT+5:30

File Name

SMART_SUPPLY_PROJECT_Final_Print.docx

File Size

5.3 MB

27 Pages

4,664 Words

23,831 Characters

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

0 AI-generated only 0%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.